Nearly Optimal Parallel Broadcast in the Plain Public Key Model

Ran Gelles[1], Christoph Lenzen[2], Julian Loss[2], and Sravya Yandamuri[2,3]

[1] Bar-Ilan University ran.gelles@biu.ac.il
[2] CISPA Helmholtz Center for Information Security
lenzen@cispa.de,lossjulian@gmail.com
[3] Duke University and Common Prefix syandamuri31@gmail.com

Abstract. Parallel Byzantine broadcast (PBC) (also known as Interactive Consistency), is a fundamental problem in distributed computing and cryptography which asks that all parties reliably distribute a message to all other parties. We give the first communication-efficient protocol for PBC in the model with plain public keys (i.e., no trusted dealer) which achieves security against an adaptive adversary that can corrupt up to $t < n/2$ parties.

Our protocol runs in total communication complexity $O(n^2\ell \log(n) + n\kappa^2 \log^4(n))$ bits to succeed with probability $1 - 2^{-\kappa}$, where ℓ is the length of a message. All prior protocols either rely on a trusted setup or require at least $O(n^3)$ communication complexity. As a stepping stone, we present a binary consensus protocol with the same resilience and success probability that sends $O(n^2\kappa \log(n) + n\kappa^2 \log^3(n))$ bits.

We achieve these results based on a highly efficient gossip procedure that implements echo operations at low cost, and might prove useful in deriving further efficient protocols relying on simple cryptographic tools.

1 Introduction

In the problem of *Byzantine broadcast (BC)* [30], a sender S distributes an input v to n parties using a distributed protocol Π. The protocol Π ensures the following two conditions in spite of up to $t < n$ corrupted parties deviating from the protocol arbitrarily: (i) *validity:* if S behaves honestly in Π (i.e., is not corrupted), all honest parties output v (ii) *agreement:* all honest parties output the same value v' (even if S is corrupted). BC is among the most fundamental problems in distributed computing and has important applications including multi-party computation (MPC), verifiable secret sharing (VSS), and more. Interestingly, however, broadcasts in these applications seldom occur in isolation. Rather, it is common for these protocols to consist of rounds where all parties P_i *simultaneously* broadcast their input v_i to every other party. For example, this kind of behaviour naturally occurs in VSS when parties accuse a dealer of cheating while sharing its secret. The resulting problem is known as *parallel broadcast (PBC)* or *interactive consistency*.

While PBC and BC can trivially be obtained from each other, this results in a substantial penalty to efficiency. Any PBC protocol must incur $\Omega(n^2 \cdot \ell)$ bits

of communication for messages of length ℓ, but naively calling n single-sender instances of any off-the-shelf BC protocol results in either $\Omega(n^3 \cdot \ell)$ bits of communication or in sub-optimal corruption tolerance. In the synchronous setting, where the protocol works in distinct rounds, it is possible to exploit the parallelism in the broadcast operations to improve this figure. Indeed, recent work has made significant progress in this direction by presenting authenticated PBC protocols with close to optimal communication complexity for $t < n/2$ adaptively corrupted parties [11]. However, these protocols rely on heavy cryptographic primitives such as threshold signatures, which require a trusted setup and strong number theoretic hardness assumptions. On the other hand, classical papers in the area [9,15,23] assume the *plain public key model* (sometimes also loosely referred to as Dolev–Yao model [16]), which includes only the most basic cryptographic primitives such as digital signatures, public key encryption, and hash functions. Protocols in the plain public key model have clear advantages over protocols using more advanced forms of cryptography. From a theoretical point of view, these protocols rely on strictly weaker complexity assumptions and avoid trusted setup. From a practical perspective, minimizing the use of complex cryptography also leads to more light-weight and efficient protocols.

Unsurprisingly, however, designing protocols in the plain public key model is significantly more challenging. In particular, no communication-optimal protocol in the plain public key model for $t < n/2$ corruptions is known even for the significantly simpler problem of *binary consensus,* in which the parties need to agree on a binary output, and validity states that if all honest parties' inputs are identical, this value should also be the output. Given that PBC directly implies consensus, it should not come as surprise that designing a communication optimal PBC protocol for messages of arbitrary length ℓ in the plain public key model is a fundamental problem that has stood open for over four decades. In this work, we close this gap by giving a novel PBC protocol Π^* which, for the first time, combines all of the following beneficial properties:

- Π^* tolerates $t < n/2$ adaptively corrupted parties.
- Π^* runs in nearly optimal communication complexity of $O(n^2\ell \log(n) + n\kappa^2 \log^4(n))$ where κ denotes the length of a digital signature and the failure probability is at most $2^{-\lambda}$.
- Π^* works in the *plain public key model*: it uses only collision resistant hash functions, digital signatures, and public key encryption. It relies neither on a trusted dealer for setup nor on heavier cryptographic primitives such threshold signatures, verifiable random functions, or SNARKs.

Our protocol Π^* builds on a novel authenticated protocol for binary consensus which itself communicates $\tilde{O}(n^2\kappa + n\kappa\lambda)$ bits and tolerates up to $t < n/2$ adaptive corruptions in the plain public key model. Our binary consensus protocol is of independent interest as it improves upon the best known authenticated protocol of Momose and Ren [28] in the plain public key model, which, for some constant $\epsilon > 0$, tolerates only $t < n/2 \cdot (1 - \epsilon)$ adaptive corruptions.

1.1 Contributions and Technical Overview

At the core of any authenticated consensus protocol, honest parties attempt to gather a sufficient number of signatures on a value in order to determine its validity as an output. The distinguishing feature of authenticated protocols (in contrast to unauthenticated ones) is that a party can forward signatures to other parties in order to convince them to output the same value. This powerful property of digital signatures allows overcoming the famous lower bound of Lamport et al. for the unauthenticated setting, which prohibits consensus when $n/3$ or more parties are corrupted [30]. Authenticated protocols, on the other hand, can tolerate up to $t < n/2$ corruptions for the problem of consensus and $t < n$ corruptions for the problems of broadcast and parallel broadcast.

The Burden of Echoing. While authenticated consensus and broadcast protocols tolerating an optimal number of corruptions have long been known, the above discussion completely overlooks any efficiency concerns. A central efficiency metric is a protocol's *communication complexity*, i.e., the number of bits parties exchange in order to reach agreement. In order to keep this number at a minimum, a challenging aspect of authenticated consensus protocols is to implement the step of forwarding a list of signatures (henceforth referred to as a *certificate*) as efficiently as possible. Indeed, it is not hard to see that a naive implementation in which parties simply echo any certificate they assemble will lead to $O(n^3 \cdot \kappa)$ bits of communication in any step of the protocol.

Unfortunately, this communication complexity is unacceptable for applications of consensus that involve thousands or even hundreds of parties. To lower communication, existing protocols have taken one of two approaches. The first approach leverages stronger cryptography such as zero-knowledge proofs, threshold signatures, verifiable random functions [21], time-locked puzzles [32], or homomorphic encryption [6]. While such protocols have been able to achieve significantly smaller communication complexities, they typically rely on strong number theoretic hardness assumptions or a trusted dealer that securely generates and distributes parameters during a setup phase. In addition, the efficiency gain in communication is arguably offset by the heavier computational load incurred by usage of these primitives. An alternative approach that has only recently been explored [29,33] is to leverage expander graph based communication in order to reduce the redundancy of protocol steps in which all-to-all communication between parties occurs. While the main advantage of protocols in this category is that they avoid use of heavy cryptography, they are often far more difficult to prove secure. As a result, existing protocols using this approach either require $\Omega(n^3 \cdot \kappa \cdot \lambda \log(n))$ bits for parallel broadcast (where λ denotes a statistical parameter associated with the degree of expansion) or are secure for a suboptimal number of parties $t < n/2 \cdot (1 - \epsilon)$.

Distributing Certificates Efficiently: A Strawman Approach. To address the issues with existing protocols, one of our key contributions is a novel routine in the plain public key model for distributing certificates efficiently. To begin, let us first examine a basic approach (taken, e.g., by the work of Tsimos et

al. [33]) for distributing a certificate far more efficiently than simply echoing it. Suppose that a party P holds a certificate C comprised of $O(n)$ signatures that it wishes to distribute to all parties. Rather than echoing the entire certificate, it could instead send the certificate to only a small number, say $\lambda \ll n$, of parties, who in turn do the same. It is not hard to verify that within $\Omega(\log(n))$ rounds of this gossip process, all parties learn C at a cost of communicating only $O(n^2\lambda\kappa\log(n))$ bits, with high probability in λ. However, this approach has an obvious drawback: it can easily be shut down by an adaptive adversary who corrupts all λ parties who have learned the message after the first step of the gossip. To overcome this limitation, P breaks C into its constituent signatures, encrypts each signature and sends it to designated individual parties. In more detail, P sends every signature in C to about λ many random parties, so that each signature is received by at least $\Omega(\lambda)$ honest parties, who can then repeat this process in the next round on the received signatures. During this process, parties keep track of the signatures that they have already forwarded and do not repeatedly forward signatures on the same message with the same signer i. If a party ever receives valid signatures on conflicting messages from a signer P_i, it simply relays both of these signatures to indicate that P_i is corrupted, and subsequently forwards no further signatures on behalf of signer P_i.

By repeating this process for $\Omega(\log(n))$ rounds, each honest party can again learn C within at most $O(n^2\lambda\kappa\log(n))$ communication, with probability $1 - 2^{-\Omega(\lambda)}$. Encryption ensures that even an adaptive adversary cannot tell who has received which signatures and use this knowledge to prevent their further propagation. In order to prevent a dishonest party P_i from introducing false signatures on many different messages and thereby preventing the propagation of its signature in C, we use the following simple idea. If parties hold signatures of P_i on conflicting messages at the end of the gossip process, they can use them as valid signature for *any* message—such signatures prove that P_i is dishonest, so it *could* have sent anything anyway. This conflicting-signatures "proof" replaces the need to distribute all signed messages that P_i may have generated.

Unfortunately, this approach is suboptimal in terms of communication efficiency, as it incurs an extra factor of λ over the (probably unavoidable) cost of $O(n^2\kappa)$, needed for each party to send its signed input to any other party. We stress that for practical systems, this is a very significant overhead, as it usually makes sense to consider the parameters n, λ, κ as polynomially related. For example, choices of $\lambda = 128$ and $\kappa = 256$ associated with the 128-bit security level would still factor into the overall communication complexity as heavy as $O(\sqrt{n})$ even for relatively large systems of size $n \approx 16{,}000$ (and much heavier for smaller ones). Thus, one of our main goals in this work is to avoid unnecessary factors of λ and κ without using heavy machinery.

An Improved Gossip Approach. As described above, the main bottleneck of our gossip approach comes from the fact that the party P holding the certificate C needs to echo each signature σ in C to λ many parties in the initial step in order to ensure that at least one honest party receives σ and can repeat the process. While this is fine in case P is the only party holding C, it incurs an unnecessary

communication overhead of λ when most parties, or maybe even all parties but one, already know C and unnecessarily echo each signature in C to λ parties.

To avoid this potential superfluous communication, we introduce a more fine-grained gossip step in which parties selectively attempt to pull only the signatures that they are missing from other parties (with some small probability for each of the missing signatures). Unfortunately, this approach does not ensure that a specific honest party learns a specific missing signature with high probability in all possible situations. The intuitive reason is that the expectations of the involved random variables do not grow enough to ensure concentration with high probability when analyzing a single signature at a time. As a remedy, our analysis argues about all the signatures *at once*. This choice increases the number of random variables we concentrate upon and hence the expectations occurring in the utilized Chernoff bounds. In this manner, we can ensure that almost all signatures in C will become known to all but a constant fraction of honest parties within roughly $O(\log(n))$ iterations, with high probability.

However, once most parties know most signatures in C, the number of random variables of missing signatures drops again, and we cannot use this concentration approach anymore. Luckily, in this case, the probability for any particular signature to be pulled is sufficiently large, allowing us to argue about each signature individually. We then show that, since most pulls are successful, the parties still missing signatures quickly learn them all.

While this informal description captures some of the intuition of our protocol, it hides some crucial technical difficulties that we must deal with. One particular issue that we have already mentioned is that a malicious signer can keep introducing new signatures into the protocol to meddle with our propagation technique. This could force parties to answer many more pull requests for a particular signer's signature than would usually be necessary. To counter this possibility, we make use of a well-known probabilistic equality test (see, e.g., [26, Ch. 3]) that allows parties to check whether they are going to pull an already-known-signature (and message) of length ℓ by communicating only $O(\log(\ell))$ bits. Thus, a party will not pull a signature unless it offers new information towards completing C.

From DistributeCertificate **to Binary Consensus.** We encapsulate the gossip procedure outlined above in a subroutine called DistributeCertificate. To obtain our binary consensus protocol, we then follow the standard methodology of first building a graded consensus protocol from DistributeCertificate. In graded consensus, a party outputs any value v along with a grade $g \in \{0, 1, 2\}$, which indicates that party's confidence in its output. In more detail, grade 0 corresponds to default output $\perp$, whereas grades 1 and 2 indicate low and high confidence in v, respectively. Crucially, the properties of graded consensus ensure that grades of honest parties differ by at most one, and that if all honest parties start with the same input v, then they all obtain the output $(v, 2)$.

Our DistributeCertificate protocol readily lends itself to building an efficient graded consensus protocol as follows. In the first step, a party signs its value and multicasts it to all parties. Each party then constructs a certificate C from the received signatures (if possible), which it distributes in parallel with other par-

ties via a couple of DistributeCertificate invocations. Parties then grade their final output according to the number of phases during which they saw no certificates for conflicting values. It can easily be verified that this number and thus the resulting grades can differ by at most one between honest parties. Importantly, each phase incurs at most $\tilde{O}(n^2\kappa + n\lambda\kappa)$ bits of communication. We then rely on the recursive phase king template of Momose and Ren [28] (adopted from the original work of Bermann et al. [5]) to obtain a binary consensus protocol with the same communication complexity. Adopting the convention that all statements refer to randomized algorithms that fail with probability at most $\mathsf{negl}.(\lambda)$, we arrive at the following result.

Theorem 1. *There is a binary consensus protocol tolerating $t < n/2$ faults, where honest parties send $O(n\lambda\kappa \log^3(n) + n^2\kappa \log^2(n)/\max\{\log(n/\lambda), 1\})$ bits.*

Corollary 1. *For $\lambda = n^{1-\Omega(1)}$, the protocol from Theorem 1 has honest parties send $O(n^2\kappa \log(n))$ bits.*

From Binary Consensus to Parallel Broadcast. In order to go from binary consensus to parallel broadcast, we develop a new recursive framework that allows distribution of all of the senders' messages with quasi-optimal communication complexity $(n^2(\ell + \kappa)) \log^{O(1)}(n)$, for messages of length ℓ. Our framework defines a binary tree $\mathcal{T}$, where the root node contains the entire set of parties and every pair of child nodes recursively splits the set of the parties in its parent node into disjoint subsets of balanced size. For simplicity, we assume for this discussion that $n = 2^k$ for some k. Thus, the leaves of $\mathcal{T}$ contain only a single party. Initially, each party multicasts their signed input, resulting in each party obtaining an input vector V of signed messages, possibly with $\perp$ entries for dishonest parties refusing to send a correctly signed message. The parties then recursively propagate and merge their vectors, where the goal is that (i) honest parties' values are maintained and (ii) increasingly larger sets of parties agree on a vector. To achieve this, the procedure progresses through $\mathcal{T}$ from the bottom up as follows:

- In the first step, parties send their current vector V to the (two) parties in their parent node in $\mathcal{T}$ (including themselves). Thus, every leaf node party obtains a second vector of signed values.
- Each pair of parties in the parents nodes of the leaves merge their vectors V_1 and V_2 of received values on all n indices. Namely, on index k, the new vector has no entry if $V_1[k]$ and $V_2[k]$ are both $\perp$; it is $V_2[k]$ if only $V_1[k] = \perp$; or otherwise it is $V_1[k]$. In other words, we drop a message if and only if the vectors hold conflicting messages signed by party P_k.
- The previous step is now repeated at each level of the tree. The difference is that internal tree nodes correspond to a *set* of parties, which by the guarantees of the recursion agree on a vector V provided that the set has an honest majority.
  - The parties in each left child node $\mathcal{L}$ collectively act as a sender in a *collective broadcast* instance (Definition 5) in order to consistently distribute the vector $V_{\mathcal{L}}$ (which itself was obtained from merging the vectors of its child nodes) to all the nodes in its parent node $\mathcal{P}$. In parallel,

the same action is taken by parties in the sibling node $\mathcal{R}$ of node $\mathcal{L}$ in the tree $\mathcal{T}$, resulting in the vector $V_{\mathcal{R}}$ distributed to all $\mathcal{P}$.
  - Parties in $\mathcal{P}$ locally merge $V_{\mathcal{L}}$ and $V_{\mathcal{R}}$ as described for the leaf level.
- At the top level, parties output their merged vector.

The scheme succeeds by the observation that at least one path from the root node of $\mathcal{T}$ to a leaf has an honest majority within every internal node on the path. See details in Section 4.4.

Towards the above, we show a communication efficient reduction of a collective broadcast operation to a collective version of *graded broadcast* (Definition 4) and the *binary agreement* (Definition 3) described above. In the collective broadcast routine, parties in $\mathcal{L}$ (say) apply standard error correction techniques to the vector $V_{\mathcal{L}}$ in order to propagate it as efficiently as possible; Using a cryptographic accumulator (Definition 10), they also send a short digest of their codeword. Parties in $\mathcal{P}$ then perform a graded agreement on the digest value received by a majority of the parties in $\mathcal{L}$. Finally, according to the resulting grade, a binary agreement is run in order to decide whether parties in $\mathcal{P}$ are able to reconstruct $V_{\mathcal{L}}$ or not. See details in Section 4.3.

In Section 4.2, we show how to construct graded broadcast from our gossiping primitive DistributeCertificate; this step crucially relies on the fact that DistributeCertificate efficiently handles a large universe of possible values for certificates, and can efficiently spread any certificate, including equivocating certificates, throughout the network.

Analyzing the resulting communication of each building block in our parallel broadcast protocol, one can verify that the communication complexity at each recursive level decreases at the same rate as the number of nodes increases. Hence, our communication suffers only a $\log(n)$ factor in cost over all of these levels. Using this approach, we obtain the following result.

Theorem 2. *There is a protocol implementing parallel broadcast that incurs*

$$O\left(n^2\ell\log(n) + n\lambda\kappa\log^4(n) + n^2\kappa\cdot\frac{\log^2(n)}{\max\{\log(n/\lambda),1\}}\right)$$

communication complexity and tolerates $t < n/2$ corruptions.

Corollary 2. *If $\lambda = n^{1-\Omega(1)}$, the protocol from Theorem 2 has communication complexity $O(n^2(\ell+\kappa)\log(n))$.*

The complexity of our protocols is near-optimal in a practical sense: if we allow for a failure probability of $2^{-\lambda}$ for, e.g., $\lambda = n^{0.8}$, the gap to the lower bound becomes $O(\log(n))$ without any large hidden constant.[4] For $n = 64$, this means that the chances for the protocol to fail are smaller than 1 in 240 billion.

We believe that our recursive template can be useful independently of our concrete construction.

[4] We did not attempt to optimize constants in this work.

1.2 Related Work

Both broadcast and consensus are fundamental and well-studied problems in distributed computing. The seminal work of Lamport, Shostak and Pease [30] introduced the problem of broadcast and gave a solution with optimal $t+1$ rounds (for deterministic protocols), yet exponential communication complexity. Their work was subsequently improved by Garay and Moses [20] who showed the first round-optimal protocol with polynomial communication complexity. In the authenticated setting, Dolev and Strong [15] presented the first round-optimal broadcast protocol tolerating $t < n$ corruptions. This work also showed a $(t + 1)$-round lower bound for deterministic broadcast. Subsequently, Dolev and Reischuk [14] proved that any deterministic broadcast protocol must send $\Omega(t \cdot n)$ messages. Deterministic protocols matching this lower bound were given independently by Berman, Garay, and Perry [5] and Coan and Welch [12] for the error-free setting and more recently by Momose and Ren [29] for the authenticated setting.

Randomized protocols are known to circumvent both of the above bounds and have been shown for various settings. King and Saia were the first to give protocols with $o(n^2)$ communication complexity in the error-free setting with $t < n/3 \cdot (1 - \epsilon)$ corruptions [25,24]. In the authenticated setting, various works have leveraged advanced cryptography to obtain subquadratic communication complexity for $t < n/2 \cdot (1 - \epsilon)$ [27,21,2]. However, these protocols rely on small, randomly selected subcommittees in order to cheaply drive consensus. Therefore, they rely on trusted setup in the form of a dealer that distributes parameters and keys of a verifiable random function and are only secure for at most $t < n/2 \cdot (1 - \epsilon)$ corruptions. Moreover, these protocols require $O(\lambda \cdot \kappa \cdot n)$ bits of communication for a single instance of (binary) broadcast, meaning that a naive n-wise composition would result in a parallel broadcast protocol with at least a factor of λ overhead in communication. We are not aware of any protocol that achieves subquadratic communication complexity when $t < n/3$ in the error-free setting or $t < n/2$ in the authenticated setting, respectively.

Many randomized protocol have also studied consensus and broadcast in terms of round-complexity. Notable examples include the ground breaking work of Feldman and Micali [17] who gave the first broadcast and consensus protocols running in $O(1)$ expected rounds for the error-free setting with $t < n/3$. Building on their ideas, Katz and Koo [23] later extended these results to the authenticated setting for $t < n/2$. A sequence of works initiated by Garay et al.[19] has also studied randomized broadcast protocols with sublinear round complexity for the setting with $t < n$ corruptions [18,10,35,36].

By comparison, parallel broadcast has only seen comparatively recent interest from the community. Starting with the work of Tsimos, Loss, and Papamanthou [33], several papers have explored communication efficient protocols for the parallel broadcast problem. Tsimos et al.'s work gives binary parallel broadcast protocols for the dishonest majority setting with $t < n \cdot (1 - \epsilon)$ corruptions that achieve $O(n^2 \cdot \lambda^3 \cdot \kappa \cdot \log(n))$ communication complexity using trusted setup and $O(n^3 \cdot \kappa \cdot \lambda \log(n))$. More recently Abraham et al. [1] gave a protocol with $O(n^2 \cdot \ell + n^2 \log(n))$ communication complexity for messages of length ℓ in the

setting of $t < n/3$. This result was recently improved by Asharov and Chandramouli [4] who gave a perfectly secure parallel broadcast protocol in the same setting which achieves $O(n^2 \cdot \ell + n^3 \log^2(n))$ bits of communication. In the authenticated setting, recent work of Abraham, Nayak, and Shrestha [3] presents a protocol tolerating $t < n/2$ corruptions with $O(n^2 \cdot \ell + n^3\kappa)$ complexity. In very recent work, Civit et al. [11] give an authenticated parallel broadcast protocol with $O(n^2 \cdot \ell + n \cdot f \cdot \kappa)$ bits of communication, where $f \leq t$ denotes the actual number of corruptions in an execution. Finally, Collins et al. [13] give a parallel broadcast protocol for the $t < n$ setting with $O(n^2 \cdot \ell + n^2\lambda\kappa)$ complexity. Compared to our work, however, their protocols heavily rely on trusted setup assumptions such as VRFs and threshold signatures, which require trusted setup.

2 Preliminaries and Notation

Network and adversary model. We consider a network of n parties in which parties are connected via point to point authenticated channels. Messages are delivered *synchronously*, where all messages sent at a given round arrive before the end of that round. We assume the *atomic send model*: if a party is honest at the time of multicasting a message, the message is delivered to all parties.

We assume a probabilistic polynomial time (PPT) adversary that may corrupt up to $t < n/2$ parties adaptively over the course of the protocol. Corrupted parties are *Byzantine*, namely, they may deviate arbitrarily from the protocol. We consider a *rushing adversary* which may choose its own messages for a round after seeing *all* honest parties' messages, i.e., it has reading access to all the network channels. However, the adversary cannot delete or corrupt messages already sent by an honest party. Note that our atomic send model assumption implies that when an honest party sends a batch of messages to multiple other parties in a single step of the protocol, it may be corrupted only before or after the entire batch of messages has been sent (but not necessarily delivered). Our model assumptions are in line with works in the area [33,6].

Model assumptions and notations. We assume that parties can securely erase their local states. We use κ to denote a computational security parameter that is associated with the size and security of the various cryptographic primitives used throughout the paper. In addition, λ denotes a statistical security parameter. We make no assumption on the relation between n, κ, and λ, except that we require (for technical reasons) that $\mathsf{negl}.(\lambda) \cdot \mathsf{poly}(\mathsf{n}) = \mathsf{negl}.(\lambda)$ and $\kappa \geq \lambda$. In this paper, $\mathsf{negl}.(x) \coloneqq 2^{-\Omega(x)}$. All our constructions are PPT algorithms that may fail with negligible probability in λ. However, for the sake of readability, we eschew error probabilities from our definitions for distributed problems related to consensus. We stress that all our definitions could easily be restated to take error probabilities into account.

Digital signatures and Certificates. We assume that parties have a public-key infrastructure available, i.e., all parties hold the same vector of public keys $(\mathsf{pk}_1, \ldots, \mathsf{pk}_\mathsf{n})$, and each party P_i holds the secret key sk_i associated with pk_i. A

party P_i can use its secret key $\mathsf{sk_i}$ to create a signature $\langle m \rangle_i$ of a message m that can be efficiently verified by anyone using $\mathsf{pk_i}$. A signature that verifies under $\mathsf{pk_i}$ is said to be *valid*; otherwise, it is *invalid*. We denote the size of of signatures and keys as κ. As is common for this line of literature (in order to reduce the amount of technical overhead), we treat signatures as information theoretic primitives, i.e., we assume that the adversary cannot forge signatures of honest parties. We remark that it would be simple to instantiate our signature schemes with any scheme satisfying unforgeability under chosen message attacks [22].

We next define *elements*, which are comprised of zero, one, or two distinct pairs of binary strings m together with a valid signature $\langle m \rangle_i$ on m under the public key of (the same) party P_i. Elements cover multiple different formats in which such pairs will be used and therefore serve as a convenient intermediate layer of abstraction in our protocols.

Definition 1 (Element). *Let $v, v' \in \{0,1\}^*, v \neq v'$ and $i \in [n]$. Moreover, let $\langle v \rangle_i, \langle v' \rangle_i$ denote (valid) signatures on v and v' with respect to the public key of party P_i. We refer to e as an* element *if e takes one of the three following formats:*

- ***Empty**: $e = \perp$*
- ***Single**: $e = (v, \langle v \rangle_i)$. We refer to v as the* payload *of e.*
- ***Double**: $e = \{(v, \langle v \rangle_i), (v', \langle v' \rangle_i)\}$. We refer to v, v' as the* payload *of e.*

For single and double elements e, we refer to P_i as the owner *of e.*

To simplify our notation, we define the operator $\oplus$ which combines elements as follows. For single elements e_1 and e_2 with the same owner P_i and distinct payloads $v \neq v'$, $e = e_1 \oplus e_2$ is defined as the double element $e = \{(v, \langle v \rangle_i), (v', \langle v' \rangle_i)\}$. If e_1 (or similarly, e_2) is a double element, $e_1 \oplus e_2$ combines the two lexicographically smaller values v_1 and v_2 from the respective payloads of e_1 and e_2 into a new double element e with payload v_1, v_2. Finally, if e is an empty element and e' is a single or double element, $e \oplus e' = e'$.

We next define *certificates*, which serve as a proof that at least one honest party has signed some value $v \in \{0,1\}^*$. A certificate consists of a collection of $t+1$ elements, all of which either have payload v or are double elements. Intuitively, a double element inside a certificate will always correspond to a dishonest owner and thus can serve as a placeholder signature on *any* value.

Definition 2 (Certificate). *A* certificate *for a value $v \in \{0,1\}^*$ is a set C consisting of $t+1$ single and/or double elements such that for any $e, e' \in C$ it holds that (1) e and e' have pairwise distinct owners i and j (2) if e (or e') is a single element, then it has payload v.*

Additional Primitives. In Appendix A we introduce additional basic primitives which are used throughout our paper, namely, Public Key Encryption (PKE), Accumulator (ACC), and Linear Erasure Correcting Codes (ECC).

2.1 Important Distributed Problems

We recall (and introduce) some important distributed problems. In the following, we distinguish between problems asking for binary and/or multivalued outputs.

For the latter, we use ℓ to denote the length of an input and (partial) output of the protocol.

Definition 3 (Binary Byzantine Agreement). *Let Π be a protocol executed by a set of parties $P_1, \ldots, P_n$ where party P_i has input $x_i \in \{0,1\}$ and terminates upon outputting a value $v_i \in \{0,1\}$. Π is a Binary Byzantine Agreement protocol if the following properties hold whenever at most $t < n/2$ parties are corrupted:*

- ***Validity:*** *If for some $v \in \{0,1\}$ all honest parties have input $x_i = v$, then all honest parties output v.*
- ***Agreement:*** *If honest parties P_i and P_j output v_i and v_j, then $v_i = v_j$.*
- ***Liveness:*** *All honest parties output a value $v \in \{0,1\}$.*

Definition 4 (Multivalued Graded Consensus). *Let Π be a protocol executed by a set of parties $p_1 \ldots p_n$ where party P_i has input $x_i \in \{0,1\}^\ell \cup \{\perp\}$ and terminates upon outputting a tuple $(v_i, g_i) \in (\{0,1\}^\ell \times \{1,2\}) \cup \{(\perp, 0)\})$. Π is a* Graded Consensus (GC) *protocol if the following properties hold whenever at most $t < n/2$ parties are corrupted:*

- ***Graded Validity:*** *If for some $v \in \{0,1\}^\ell$, all honest parties have input $x_i = v$, then each honest party outputs $(v, 2)$.*
- ***Graded Agreement:*** *If an honest party P_i outputs $(v_i, 2)$, then all honest parties P_j output (v_j, g_j) s.t. $g_j \in \{1,2\}$ and $v_j = v_i$.*
- ***Liveness:*** *All honest parties output $(v, g) \in (\{0,1\}^\ell \times \{1,2\}) \cup \{(\perp, 0)\})$.*

We now introduce a novel type of consensus problem that we refer to as *collective broadcast*. In collective broadcast, a subset of parties $S \subset \{p_1, \ldots, p_n\}$ collectively acts as a sender that distributes its values consistently to the other parties in the network.

Definition 5 (Multivalued Collective Broadcast (CBC)). *Let Π be a protocol executed by a set of parties $P_1, \ldots, P_n$, parameterized by a set $S \subseteq \{P_1, \ldots, P_n\}$ of sending parties. Every honest party $P_i \in S$ holds an input $x_i \in \{0,1\}^\ell$ and terminates upon outputting a value $v_i \in \{0,1\}^\ell$. Π is a collective broadcast protocol if the following properties hold whenever at most $t < n/2$ parties are corrupted:*

- ***Validity:*** *If a majority of the parties in S are honest, and for all honest parties in S, $x_i = v$, then all honest parties output $v_i = v$.*
- ***Agreement:*** *If an honest party P_i outputs v_i and another honest party P_j outputs v_j, then $v_i = v_j$.*
- ***Liveness:*** *All honest parties output a value $v \in \{0,1\}^\ell$.*

Definition 6 (Multivalued Parallel Broadcast). *Let Π be a protocol executed by a set of parties $P_1, \ldots, P_n$ where party P_i has input $x_i \in \{0,1\}^\ell$ and terminates upon outputting a vector $V_i \in \left(\{0,1\}^\ell\right)^n$ of n values. Π is a* Parallel Broadcast *protocol if the following properties hold whenever at most $t < n/2$ parties are corrupted:*

- ***Validity:*** *If P_j is honest, $V_i[j] = x_j$ for all honest parties P_i.*
- ***Agreement:*** *If honest parties P_i and P_j output V_i and V_j, then $V_i = V_j$.*

– ***Liveness:*** *All honest parties output a vector* $V \in \left(\{0,1\}^\ell\right)^n$.

We note that all of our protocols have known running time bounds. Thus, liveness is trivial to achieve, even if recursive or subroutine calls are used: each honest party proceeds with the execution exactly after the allotted number of rounds is over. Accordingly, in the following liveness will tacitly be assumed without separate discussion.

3 Gossiping and Distribute Certificate

In this section we present our protocol DistributeCertificate. As indicated by the name, DistributeCertificate is used to distribute certificates on one or more values.

3.1 Gossiping

At the heart of our protocol lies an efficient gossip procedure, which fails only with probability $\mathsf{negl}.(\lambda)$. Its goal is that, if an honest party inputs a certificate for value v and survives until the end of the routine, all honest parties will hold a certificate for v. Our routine might fall slightly short of that goal in case there are multiple certificates that can be formed, which is possible e.g. if the sender of a broadcast operation is dishonest. However, if a party does not obtain a certificate for v in such a scenario, we guarantee that it obtains certificates for at least two distinct values. As this implies that the sender is dishonest, we are free to interpret this situation as the sender having provided a certificate for *any* possible value, including v, formally recovering the desired property that a certificate for v is obtained by each party.

To be communication-efficient in spreading certificates, we want that each pair of parties exchanges only a constant number of elements in each step. The idea of using double elements, i.e., conflicting signatures by the same owner, as stand-in for signing *any* possible value means that for each index, contribution to a certificate for value v spreads unimpeded by communication of signatures for different values v'.

This naive gossiping approach means that every party sends each one of the n elements of the certificate C it holds to $O(1)$ random parties, resulting in only a constant probability of success in spreading C throughout the network. To see that, consider the case where only a single honest party P_i holds a signature required for a certificate C owned by (dishonest) P_k. The successful event is when P_i sends this signature to some other honest parties, who then spread it to more honest parties, exponentially increasing the number of (new) parties learning this signature, until after $O(\log(n))$ steps most honest parties know it. Subsequently, the remaining honest parties learn the signature within another $O(\log(n))$ steps, w.h.p. However, we could get unlucky in that P_i never selects an honest party to spread the signature to. Doing the math, the success probability for each individual signature is constant, which falls short of the desired $1 - \mathsf{negl}.(\lambda)$ bound.

A way out, which works in the regime where λ is at most (roughly) n, is to reason about all owners P_k concurrently. We may not succeed in spreading *all*

of the n signatures in one go, but we can show that with probability $1 - \mathsf{negl}.(n)$ a constant fraction of the signatures is spread. We then introduce a mechanism to boost the probability of spreading individual signatures for the few remaining signatures that are not yet known to all parties: If x signatures still need to be distributed, each party in each step can sample $\Theta(n/x)$ other parties and send them each, all x signatures. This boosts the probability that the gossip succeeds on a given signature from constant to $1 - 2^{\Theta(n/x)}$, which in turn means that a much larger fraction of the remaining signatures gets delivered. The process then recursively repeats, putting more and more effort on the still-missing signatures without increasing the per-round communication. A careful analysis of this procedure shows that $O(\log^2(n)/\log(n/\lambda))$ gossip steps are sufficient to deliver all required elements with probability $1 - \mathsf{negl}.(\lambda)$.

To implement this approach, we need to navigate one more obstacle. Parties are not aware of the indices k for which the gossip already succeeded, implying that P_i cannot determine on its own on which indices to focus its efforts. The solution here is to first check with the receiving party P_j which signatures it is still missing, and communicate only these respective elements. Naturally, doing so for all indices is too costly. However, communicating additional $\lambda \leq \kappa$ bits per party will not change the asymptotic complexity, which means that every party can exchange information about $\lambda/\log(n)$ indices. Specifically, for any specific index that P_i considers to retrieve from P_j, it can send $O(\log(n))$ bits in order to determine whether or not the information held by P_j for that index is useful for P_i. This solution leverages randomized equality testing (Appendix B) to detect if P_i and P_j hold *different* values with valid signatures by the same owner P_k, i.e., detect equivocation.[5]

It turns out that testing $\lambda/\log(n)$ random indices k (i.e., owners P_k) is just enough. Intuitively, if a single index is left to gossip, sampling that many indices yields a probability of $\Theta(\lambda/(n\log(n)))$ to spread the respective signature from P_i to P_j. Over $\log(n)$ iterations, the expected number of parties P_i shares the signature with (if they do not already know it) becomes $\Theta(\lambda)$, which suffices to ensure success with probability $1 - \mathsf{negl}.(\lambda)$.

In the next sections, we formalize our gossip step as the abstract functionality $\mathcal{F}_{\mathsf{pull}}$ (Fig. 1) and provide an efficient implementation for it, Pull (Fig. 4), whose security properties are proven in Appendix C. Based on this, in Appendix D we analyze our DistributeCertificate protocol (Fig. 5) and prove the following.

Theorem 3. *Suppose that $\lambda < cn$ for a sufficiently small constant $c > 0$ and* DistributeCertificate *runs for $C\log^2(n)/\log(n/\lambda)$ rounds for a sufficiently large constant C. Then,*

(i) no party obtains any signature/value pair valid for an honest party that was not already present in some party's input and

[5] This requires $\Omega(\log(\ell))$ bits for ℓ-bit values, but if ℓ is large compared to $\log(n)$, the communication complexity is dominated by the cost of communicating the values.

(ii) with probability $1-\mathsf{negl}.(\lambda)$*, if an honest party inputs a certificate for value* v *and is not corrupted before* DistributeCertificate *completes, then each honest party outputs a certificate for* v *or two certificates for distinct values.*

Moreover, denote by ℓ *the number of bits in an element. If* $\mathcal{F}_{\mathsf{pull}}$ *is implemented by* Pull*, honest parties send in total* $O(n^2(\ell+\kappa)\log^2(n)/\log(n/\lambda))$ *bits of communication.*

As we will call DistributeCertificate recursively, in particular on sets of far fewer than n parties, the restriction of $\lambda < cn$ is problematic for guaranteeing success of the calling protocol with probability $1-\mathsf{negl}.(\lambda)$. We resolve this by a simple boosting argument using parallel repetition, leading to the following lemma, whose proof can be found in the appendix.

Lemma 1. *Suppose that* $\lambda \in \Omega(n)$*. There is a protocol with the same properties as in Theorem 3 and communication complexity of* $O(n\lambda(\ell+\kappa)\log^2(n))$*.*

3.2 The Ideal Functionality $\mathcal{F}_{\mathsf{pull}}$

In order to specify DistributeCertificate more concisely, we abstract a large part of its internal machinery related to gossiping elements into a functionality $\mathcal{F}_{\mathsf{pull}}$, which we describe in more detail before giving DistributeCertificate. Intuitively, $\mathcal{F}_{\mathsf{pull}}$ takes as input from every party P_i two lists K_i and M_i of n elements each. These lists indicate which elements P_i is willing to propagate and all the elements it knows, respectively. The latter is essential for efficient information dissemination—parties use them to "find" elements useful to gossiping, while communicating only a small number of bits per candidate element—but care must be taken not to leak information about which party has which elements before the elements are propagated to a new (and secret!) set of parties.

To this end, for an ordered pair of parties (P_i, P_j), P_i takes on the role of the sender and P_j the role of the receiver of new information. Party P_j secretly samples $c = O(1)$ independently and uniformly random "query lists" of $\lceil \lambda/\log(n) \rceil$ indices, and for each such list L it tests whether there is some index $L[k]$ such that $K_i[L[k]]$ is more useful towards constructing a certificate of interest than $M_j[L[k]]$. Intuitively, the more different signatures per element, the better. This is facilitated by the randomized predicate OrderRel that parties use to determine which elements provide important new information.

Disclosing Information in Two Steps. In the first step of $\mathcal{F}_{\mathsf{pull}}$, the adversary may gain comprehensive knowledge on the receivers' lists M_j, but learns nothing new about the senders' lists K_i. Accordingly, $\mathcal{F}_{\mathsf{pull}}$ leaks all M_j right away, constructs the query lists, and then proceeds to the second step. In the second step, each party P_i shares, for each of the c query lists from P_j, up to one element with P_j. More specifically, from each of the query lists, P_i shares with P_j the element specified by the first entry of $L[k]$ such that OrderRel indicates the element $K_i[L[k]]$ to be of use to P_j. After all honest parties have performed this operation, the honest parties output what they have learned, aggregated into the sets S_j. Note that the adversary is free to provide elements as

well via RespondDirect, at any time before the outputs are determined. This does not interfere with the intended functionality, as (i) the adversary can only add to the output sets S_j and (ii) elements are verified by signatures, so that no spurious output can be generated.

It is important to note that the adversary can use the information that is shared with corrupted parties in the second step to learn about honest parties' lists K_i. Looking forward, it will be pivotal that our implementation Pull of $\mathcal{F}_{\mathsf{pull}}$ bundles the operations of this step into a single broadcast. This way, we can exploit the model assumption of atomic sends with no take-backs. Accordingly, the adversary learns K_i only after P_i spreads its knowledge. Since the adversary obtains no further knowledge about parties' sampled lists, it does not learn which of the elements in K_i ultimately end up in which party P_j's output S_j.

Technical Description of $\mathcal{F}_{\mathsf{pull}}$. We now explain the functionality $\mathcal{F}_{\mathsf{pull}}$ in technical detail. As we have already explained, each party P_i propagates elements in its list K_i randomly to parties P_j who do not already hold these elements in their respective versions of list M_j. This way, every party invokes $\mathcal{F}_{\mathsf{pull}}$ as both a sender and as a receiver. In our description of $\mathcal{F}_{\mathsf{pull}}$, we use the index i to denote a party P_i when it is acting in the role of a sender propagating elements from its list K_i. Similarly, we use index j to denote a party P_j that is acting in the role of a receiver, pulling random indices from the lists K_i of other parties P_i that are not contained in P_j's list M_j. Whether or not to propagate an element from P_i's list K_i to another party P_j is determined by $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}}$ via the binary relation $\mathsf{OrderRel}$. Intuitively, $\mathsf{OrderRel}$ detects whether or not the pulling party already knows the element,[6] and if so, indicates that it should not be exchanged again.

$\mathcal{F}_{\mathsf{pull}}$ keeps track of a party P_i's random samples from its input list K_i that should be propagated via the two dimensional array R_i. Per party P_j, c samples of size $\lceil \lambda / \log(n) \rceil$ from P_i's list K_i will be propagated. This is done to increase the probability that P_i could send P_j an element that P_j does not already hold in their list M_j. These samples are kept in the fields $R_i[\gamma][j], \gamma \in [c]$, of P_i's array.

Once all honest parties have input lists K_i and M_i, $\mathcal{F}_{\mathsf{pull}}$ enters into a (nested) loop during which the lists $S_{j,i}$ are written. These lists are finally combined in order to determine P_j's output S_j, i.e., the combination of all new elements that it managed to obtain from other parties. In more detail, list $S_{j,i}$ stores, at position k, the element owned by P_k that P_j newly obtained from P_i.

In the outer of the two loops, the parties P_i from whose lists K_i the elements should be propagated are chosen in adversarial order. This is captured implicitly by leaking to the adversary the list K_i (thereby passing the activation token back to the environment) at the end of each iteration, after all elements from P_i have been propagated.

In the inner loop, every other honest party $P_j \neq P_i$ is sent the first element k from each of the c random samples from K_i for which $\mathsf{OrderRel}$ determines that M_j does not already contain it. This is done by writing this element (if it exists)

[6] Recall that double elements are considered to sign *any* possible element and are hence interchangeable. Thus, if the receiving party P_j holds a double element owned by party P_k, $\mathsf{OrderRel}$ will always indicate that P_j knows the element.

Functionality 1: $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}}$

$\mathcal{F}_{\mathsf{pull}}$ is parameterized by the number of parties n, an integer λ, an integer $c > 0$, and publicly verifiable PPT predicate $\mathsf{OrderRel}$. $\mathsf{OrderRel}$ is randomized and takes as input two elements. It returns 0 or 1 and uses fresh random coins on each invocation. $\mathcal{F}_{\mathsf{pull}}$ manages the following internal variables. Throughout the description, we make the convention that P_i denotes a party who is propagating elements whereas P_j is a party that is receiving elements.

- S_j is a list of new elements that P_j learns from invoking $\mathcal{F}_{\mathsf{pull}}$ that is initialized to the empty list. Similarly, $S_{j,i}$ for $j \in [n]$ is a list of new elements that P_j learns from P_i that is initialized to the empty list
- R_j is a c by n by $\lceil \lambda / \log(n) \rceil$ array initialized to $\perp$ in each entry corresponding to the requests of P_j
- K_i stores a list of n elements P_i is willing to supply to other parties and is initialized to the empty list
- M_j stores a list of n elements that P_j admits to already holding and is initialized to the empty list

On input (M, K) from P_j, where $M \subset [n]$ is of size at most n and K is of size at most n:

- Set $M_j = M$, $K_j = K$ and output M_j to adversary
- For $\gamma = 1 \dots c$ and $i = 1 \dots n$:
  - Let $R'_{\gamma,i}$ be a uniform permutation of $[n]$, truncated to length $\lceil \lambda / \log(n) \rceil$
  - Let $R_j[\gamma][i] = R'_{\gamma,i}$ // the γth list of indices P_j queries P_i for

Once all honest parties have input (M, K):

- For all honest P_i (in order of activation): // P_i is in the sender role
  - For all honest P_j and all γ: // this step happens atomically
    - Let $L = R_j[\gamma][i]$ denote the γth list of indices which P_j queries from P_i
    - If $\exists k$ s.t. $\mathsf{OrderRel}(K_i[L[k]], M_j[L[k]]) = 1$, pick the minimal such k and set $S_{j,i}[L[k]] = K_i[L[k]]$
  - Output K_i to adversary
- For all honest P_j: // P_j is in the receiver role
  - For all $k = 1 \dots n$, set $S_j[k] := \bigoplus_{i=1}^{n} S_{j,i}[k]$, return S_j

On input $(\textsc{RespondDirect}, e, k, j)$ by adversary (on behalf of corrupted party P_i):

- If the element e is owned by party P_k, set $S_{j,i}[k] = S_{j,i}[k] \oplus e$

Fig. 1. The ideal functionality $\mathcal{F}_{\mathsf{pull}}$

to $S_{j,i}$ at the appropriate position. We stress that the inner loop is always run in an order determined by the functionality, i.e., outside of the adversary's control, and no corruptions are possible during its execution. The adversary also gets the option to update a position k of the list an honest party P_j receives from corrupted P_i, i.e., $S_{j,i}[k]$, with an element e owned by P_k of its choosing.

3.3 Instantiating $\mathcal{F}_{\mathsf{pull}}$

In this section, we provide an instantiation of $\mathcal{F}_{\mathsf{pull}}$, which we refer to simply as Pull. Pull internally runs an interactive two-party protocol Order in order to determine whether or not to exchange elements between two parties in the protocol. Order can be instantiated in two different ways, namely via protocols Order_1 or Order_2 (see below). Therefore, we denote Pull with one fixed version of Order as $\mathsf{Pull}^{\mathsf{Order}}$. Looking ahead, each Order_i will induce a corresponding order relation $\mathsf{OrderRel}_i$ such that we can instantiate $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}_i}$ with $\mathsf{Pull}^{\mathsf{Order}_i}$. In the following, we first describe the high-level input/output properties of the placeholder protocol Order before explaining the specific behavior of Order_1 and Order_2 in more detail.

A Probabilistic Equality Test on Elements. Intuitively, Order implements a probabilistic equality check on elements in the respective lists K_i and M_j of two protocol parties P_i and P_j. Recall from our description of $\mathcal{F}_{\mathsf{pull}}$ above that P_j is acting in the role of a receiver, requesting to obtain signatures from party P_i who is acting as the sender. P_j's objective is to learn new elements from P_i's list K_i, i.e., elements which it does *not already hold* in its list M_j.

Order is a 2-round asynchronuos protocol with two subroutines $\mathsf{OrderIn}$ and $\mathsf{OrderOut}$. $\mathsf{OrderIn}$ is run by P_j and takes as input an element e owned by a party P_k. Its output consists of a single message $\tilde{m}$ which is sent to party P_i. Upon receiving P_j's message $\tilde{m}$, P_i inputs $\tilde{m}$ and the element e' it holds (with the same owner P_k) to $\mathsf{OrderOut}$ which outputs a bit b. Intuitively, b indicates whether or not e' is useful to P_j (who already holds the element e) toward completing a certificate on some value v. We stress that we deliberately choose to describe Order as an asynchronous protocol. That is, P_i can complete its respective part $\mathsf{OrderOut}$ of the protocol at any point in time upon receiving P_j's message produced in $\mathsf{OrderIn}$. This will be used in Pull, where P_i receives the respective messages not immediately, but with some delay.

Instantiating Order. As explained above, Order is instantiated via the protocols Order_1 and Order_2, depicted in Figures 2 and 3, respectively. These routines internally use a randomized two-party equality testing protocol provided by Corollary 5, which may fail to detect inequality with probability $1/n$, but always correctly asserts equality. They differ marginally in the type of test they implement.

More specifically, for the following descriptions of Order_1 and Order_2, let $(\mathsf{EqIn}, \mathsf{EqOut})$ denote a two-party equality testing protocol where the syntax is as in Corollary 5. The first part in both Order_1 and Order_2 is the same: party P_j runs EqIn on its current element for index k, and sends a message to P_i indicating whether it has an empty, single, or double element, and the output of

Order_1

The description describes the protocol as run by parties P_i and P_j.

Subprotocol $\mathsf{OrderIn}_1$ **for party** P_j:
Input: an element e owned by P_k for some k

$$\textbf{Output} = \begin{cases} (0, \perp) & \text{if } e = \perp \\ (1, \mathsf{EqIn}(v)) & \text{if } e = (v, \langle v \rangle_k) \text{ is single} \\ (2, \perp) & \text{if } e \text{ is double,} \end{cases}$$

where $\mathsf{EqIn}(v)$ is the message sent in by the protocol from Corollary 5 run between parties P_j and P_i, on the input v of P_j. Send the output to P_i.

Subprotocol $\mathsf{OrderOut}_1$ **for party** P_i:
Input: an element e' owned by P_k for some k and a tuple (x, y) received from P_j (upon completing $\mathsf{OrderIn}_1$)

$$\textbf{Output} = \begin{cases} 1 & \text{if } e' \text{ is single and } x = 0 \\ 1 & \text{if } e' = (v', \langle v' \rangle_k) \text{ is single, } x = 1, \text{ and } \mathsf{EqOut}(v', y) = 0 \\ 1 & \text{if } e' \text{ is double and } x < 2 \\ 0 & \text{else} \end{cases}$$

where $\mathsf{EqOut}(v', y)$ is the output of the protocol given by Corollary 5 run between parties P_j and P_i, on input v' of P_i, in which P_j sends the message y.

Fig. 2. Order_1, an instantiation of Order

EqIn. Then, depending on the types of element P_j and P_i hold, and the equality-testing output of EqOut on the information received from P_j, party P_i outputs a bit b, whose semantics depend on which Order is used and is explained next.

Order_1 checks whether for the owner P_k in question (i) P_i has a single element e', whereas P_j's element e is empty, (ii) both parties have single elements e, e' with different respective payloads v and v', or (iii) P_j's element e is not double, but P_i's element e' is. This serves the purpose of P_j learning a new signature owned by P_k toward a complete certificate for *any* possible value v.

Order_2 has a slightly different goal for P_j: learning a new element e' toward a certificate for a *specific* value a_i for which P_i has a certificate that it wants to propagate. Thus, here P_i only considers elements that are useful toward completing a certificate for a_i. If $a_i \neq \perp$, we say that P_i *advertises* the value a_i in the respective call to Order_2. Order_2 checks for (i) and (ii) as above, but for (iii), it additionally requires that P_j does not already hold a single element with payload a_i. To make the dependency on the values a_i explicit, we write $\mathsf{Order}_2^{\vec{a}}$, where $\vec{a}$ is a vector containing the value a_i of honest party P_i in position i. Note that it suffices if P_i knows a_i to perform the necessary evaluations. In particular, in our protocol, no honest party actually knows the exact vector $\vec{a}$.

The Pull **protocol.** With the above in place, we can now realize $\mathcal{F}_{\mathsf{pull}}$ via the protocol Pull. More specifically, we will show that $\mathsf{Pull}^{\mathsf{Order}_i}$ instantiates $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}_i}$. Here, $\mathsf{OrderRel}_i$ is the natural binary (randomized) predicate induced by Order_i

$\mathsf{Order}_2^{\vec{a}}$

The description describes the protocol as run by parties P_i and P_j. Moreover, it assumes a fixed vector $\vec{a}$ such that every honest party P_i holds the value a_i.

Subprotocol $\mathsf{OrderIn}_2$ **for party** P_j:
Input: an element e owned by P_k for some k

$$\textbf{Output} = \begin{cases} (0, \bot) & \text{if } e = \bot \\ (1, \mathsf{EqIn}(v)) & \text{if } e = (v, \langle v \rangle_k) \text{ is single} \\ (2, \bot) & \text{if } e \text{ is double,} \end{cases}$$

where $\mathsf{EqIn}(v)$ is the message sent by the protocol from Corollary 5 run between parties P_j and P_i, on the input v of P_j. Send the output to P_i.

Subprotocol $\mathsf{OrderOut}_2^{\vec{a}}$ **for party** P_i:
Input: an element e' owned by P_k for some k and a tuple (x, y) received from P_j (upon completing $\mathsf{OrderIn}_1$).

$$\textbf{Output} = \begin{cases} 1 & \text{if } e' \neq \bot \text{ and } x = 0 \\ 1 & \text{if } e' \neq \bot,\ x = 1, \text{ and } \mathsf{EqOut}(a_i, y) = 0 \\ 0 & \text{else} \end{cases}$$

where $\mathsf{EqOut}(a_i, y)$ is the output of the protocol given by Corollary 5 run between parties P_j and P_i, in which P_j sends the message y.

Fig. 3. $\mathsf{Order}_2^{\vec{a}}$, parametrized by a vector $\vec{a}$ of advertised values

which internally emulates $\mathsf{OrderIn}_i$ and $\mathsf{OrderOut}_i$ on the respective inputs of both parties and outputs the output bit of $\mathsf{OrderOut}_i$. The construction of Pull assumes a Public Key Encryption scheme stated in Definition 8.

For a concrete choice of Order, the protocol proceeds as follows. First every pair of parties P_i, P_j exchanges fresh public keys for encryption. This is necessary, because we delete keys right after using them, so that corruption of a party (and hence the adversary learning any stored private keys) does not leak too much information to the adversary.

In its role as receiver, party P_j samples for each party P_i the c lists of $\lceil \lambda / \log(n) \rceil$ indices which will be tested for "elements of interest" via $\mathsf{OrderIn}$. For each such index $k = R_{\gamma,j}$, it computes $\mathsf{OrderIn}(M_j[k])$. It encrypts all of these pairs of indices and associated $\mathsf{OrderIn}$ messages under the first public key of P_i and sends the resulting ciphertext to P_i as a message $\langle \mathsf{Request}, \dots \rangle$. Before sending, for technical reasons related to the security proof, it erases the (plaintext) lists and $\mathsf{OrderIn}$ messages from memory.

In its role as sender, party P_i then receives all $\langle \mathsf{Request}, \dots \rangle$ messages sent to it by honest parties (as well as those the dishonest parties choose to send). It decrypts them and determines the outputs of $\mathsf{OrderOut}$ corresponding to each party P_j from which it received a $\langle \mathsf{Request}, \dots \rangle$ message that decrypts successfully. More precisely, let $k = R_\gamma[m]$ be the m-th index listed in the γ-th list that P_i receives from P_j, and recall that it comes together with $\mathsf{OrderIn}(M_j[k])$ (provided that P_j is honest). P_i then computes $\mathsf{OrderOut}(K_i[k], \mathsf{OrderIn}(M_j[k]))$. For

$\mathsf{Pull}^{\mathsf{Order}}$ **for party** P_i

We describe the view of party P_i. Let M_i and K_i be arrays of n indexed elements which P_i inputs to Pull. Initialize $Reqs$ to an empty c by n by $\lceil \lambda / \log(n) \rceil$ array, $Resps$ to an empty n by c array, and S_i to an empty list. For $\gamma \in [c]$ and $j \in [n]$, initialize $\mathsf{Ord}_{\gamma,j}$ to an empty array of length $\lceil \lambda / \log(n) \rceil$. Let $R_{\gamma,j}$ be a uniform permutation of $[n]$, truncated to length $\lceil \lambda / \log(n) \rceil$. Finally, for all $k \in [\lceil \lambda / \log(n) \rceil]$ run $\mathsf{OrderIn}$ on input $M_i[R_{\gamma,j}[k]]$ and store the output in $\mathsf{Ord}_{\gamma,j}[k]$.

- **Step 1:** For $j \in [n]$, $k \in [2]$: set $(sk_{k,j}, pk_{k,j}) \leftarrow \mathsf{KeyGen}(1^\kappa)$ and send $\langle \textsc{ReportKey}, pk_{1,j}, pk_{2,j} \rangle$ to P_j
- **Step 2:** If $\langle \textsc{ReportKey}, pk_{1,i,j}, pk_{2,i,j} \rangle$ was received from P_j in Step 1, let $Reqs[\gamma][j] = \mathsf{ENC}(pk_{1,i,j}, (R_{\gamma,j}, \mathsf{Ord}_{\gamma,j}))$ and erase $R_{\gamma,j}$ and $\mathsf{Ord}_{\gamma,j}$ from memory. For all j, send $\langle \textsc{Request}, Reqs[1][j], \ldots, Reqs[c][j] \rangle$ to P_j.
- **Step 3:** If the message $\langle \textsc{Request}, \ldots \rangle$ was received from P_j in Step 2, decrypt it using $sk_{1,j}$ and let $R_\gamma, \mathsf{Ord}_\gamma, \gamma \in [c]$ denote the resulting decryption.
  For $\gamma \in [c]$:
  - If $\exists k$ s.t. $\mathsf{OrderOut}(K_i[R_\gamma[k]], \mathsf{Ord}_\gamma[k]) = 1$: Pick the minimal such k and set $Resps[j][\gamma] = \mathsf{ENC}(pk_{2,i,j}, (K_i[R_\gamma[k]]))$
  - **else** set $Resps[j][\gamma] = \mathsf{ENC}(pk_{2,i,j}, \textsc{NoMsg})$
  - erase $sk_{1,j}$ and $R_\gamma, \mathsf{Ord}_\gamma$ for all $\gamma \in [c]$ from memory

  For every response generated, send a message $\langle \textsc{Response}, Resps[j][1], \ldots, Resps[j][c] \rangle$ to the corresponding party P_j.
- **Output Determination:** If a message $\textsc{Response}$ from P_j was received in Step 3, decrypt it, store the response list in the set $Resps_{in}$. Erase any remaining secret keys from memory.
  For $r \in Resps_{in}$ and $\gamma \in [c]$:
  - For each non-empty element e in r with owner P_m, set $S_i[m] = S_i[m] \oplus e$
  - Erase r from memory

  Return S_i

Fig. 4. The Pull protocol: A secure instantiation of $\mathcal{F}_{\mathsf{pull}}$.

a list R_γ for which $\mathsf{OrderOut}(K_i[k], \mathsf{OrderIn}(M_j[k])) = 1$ for *some* $k = R_\gamma[m]$, let m be the minimum index with this property. P_i then encrypts and sends to P_j the value $K_i[R_\gamma[m]]$. Here, it is crucial that P_i pads its $\langle \mathsf{Response}, \ldots \rangle$ by encrypting and sending $\textsc{NoMsg}$ for each γ for which no such m exists. This way, the length of the $\langle \textsc{Response}, \ldots \rangle$ from P_i to P_j is fixed regardless of the encrypted content, leaking no information to the adversary. Note that in this step, it is also essential for each P_i to send all its messages together. This allows to leverage the assumption of atomic sends so that the messages to different parties provide cover traffic for each other. This prevents the adversary from corrupting P_i based on

some of the elements it spreads before it can complete the send operation.

In contrast to the erasure of the secret key $sk_{1,j}$, which again has technical reasons, it is pivotal (even in a non-technical sense) that P_i erases, for each P_j, the decoded lists R_γ, Ord_γ *before* sending its messages for this step. Failing to erase these values would allow the adversary to break our protocol in a concrete sense as follows. It first delivers the messages from P_i to an already corrupted subset of parties. By decrypting these messages, the adversary can learn near complete information on K_i. In particular, (by crafting appropriate requests of corrupted parties) it can deterministically learn, for a given index k, all honest parties P_i that hold a single or double element owned by P_k. To prevent any spread of P_k-owned elements, it now does as follows. First, it corrupts all of the honest parties who currently hold P_k-owned elements. Then, it deduces from the (non-erased and unencrypted) request lists to which additional honest parties P_j the elements owned by P_k have been sent and corrupts these parties as well. Note that this is easily feasible in the early gossip stages, in which only few honest parties hold such an element owned by P_k. This breaks the security guarantees Pull needs to uphold and lets the adversary completely suppress the spread of elements owned by P_k.

Finally, the responses are received and decrypted, all elements owned by party P_k are bundled up using the $\oplus$ operator (keeping up to two distinct values signed by P_k), and again technical reasons prompt us to erase all stored values but the output S_i from memory.

In Appendix C we prove that the protocol Pull securely instantiates $\mathcal{F}_{\mathsf{pull}}$ in MPC framework [8], when parametrized with appropriate order relation functions corresponding to Order_1 and Order_2, respectively.

The communication bounds are relatively straightforward. Step 1 requires $O(n^2\kappa)$ bits for key exchange, Step 2 uses $O(n^2\lceil\lambda/\log(n)\rceil(\log(\ell)+\log(n))$ bits for encoding the index lists and associated messages for equality testing, and in Step 3 $O(n^2(\ell+\kappa))$ bits are sent to communicate elements. As we assume that $\lambda \leq \kappa$, this adds up to $O(n^2(\kappa+\ell))$ if $\ell \leq n^2$. Otherwise, recall that Theorem 3 assumes that $\lambda = O(n)$, implying that $O(n^2\lceil\lambda/\log(n)\rceil(\log(\ell)+\log(n)) = O(n^3\log(\ell))$, which then is dominated by the $O(n^2\ell)$ term. Thus, we arrive at an asymptotic bound of $O(n^2(\kappa+\ell)$ exchanged in this case as well. Finally, to complete Theorem 3, it remains to realize $\mathsf{DistributeCertificate}$ with $O(\log^2(n)/\log(n/\lambda))$ calls to $\mathcal{F}_{\mathsf{pull}}$.

3.4 The DistributeCertificate Protocol

We now describe the $\mathsf{DistributeCertificate}$ protocol. Recall that a certificate for a value v consists of $t+1$ elements owned by distinct parties, each of which is either a single element with value v or a double element. Intuitively, the protocol seeks to achieve unimpeded spread for each of these elements, so that a certificate held by some honest party becomes known to all honest parties within $O(\log(n))$ gossip steps. To prevent an adaptive adversary from selectively corrupting all parties who know a specific element early on in the gossip process

(thereby preventing their propagation), while simultaneously maximizing the spread of new information, we rely on $\mathcal{F}_{\mathsf{pull}}$ for each step of gossip. Recall that the crucial idea behind $\mathcal{F}_{\mathsf{pull}}$ (or more precisely, its implementation Pull) to prevent an adaptive adversary from selectively hindering the propagation of certain elements is to send the same amount of encrypted bits to every party in every step. Taken over several rounds, this can be seen as running many instances of gossip *simultaneously*, where all of the messages in these instances are encrypted. In this manner, the different instances provide a form of cover traffic for each other. Roughly speaking, each of these instances will be used to propagate a small number of randomly chosen elements that make up the certificate. As specified in earlier sections, these elements are chosen carefully via $\mathcal{F}_{\mathsf{pull}}$ from the list K_i of each sender so as to maximize the spread of new (and therefore useful) information in every gossip step. There are several obstacles to this approach:

1. The concurrent gossip procedures for different owners generate congestion for each other. We address this by prioritizing the spread of new information. This, however, is exactly what is internally achieved in $\mathcal{F}_{\mathsf{pull}}$ via the sampling of index lists and the use of equality testing in the form of $\mathsf{OrderRel}$.
2. This solution introduces the following new issue. In the r-th step of gossip in $\mathsf{DistributeCertificate}$, a naive attempt might be to let each party P_i invoke $\mathcal{F}_{\mathsf{pull}}$ on all of the values that it currently holds. This list is supplied to $\mathcal{F}_{\mathsf{pull}}$ as list A'_i (for argument M_i). However, note that A'_i is immediately leaked to the adversary by $\mathcal{F}_{\mathsf{pull}}$ and includes the values that P_i received in the $(r-1)$-th step of gossip. This is problematic, as an adaptive adversary could selectively corrupt all parties who currently hold a certain element e, thereby preventing its further spread. This is resolved in $\mathsf{DistributeCertificate}$ as follows: instead of updating the list A'_i with elements that are newly received in gossip step $r-1$ right away, i.e., in gossip step r, we update A'_i with such elements only gossip step $r+1$. By this time, the elements have already spread to more parties who did not know them in gossip step r. This is because P_i still supplies an updated list of values A_i as the argument K_i to $\mathcal{F}_{\mathsf{pull}}$, so that some other parties who have not yet received these elements, learn them in gossip step r.
3. A dishonest sender can sign different values, which a dishonest party can then countersign in its role as owner to create congestion on the level of a single owner index. However, this proves equivocation by the sender to any party receiving two such signed values, allowing us to use double elements as part of certificates of *any* value, without causing issues when the sender is honest.
4. The adversary could continuously feed new double elements to (some) honest parties, causing congestion for the spread of certificate(s) for which already single elements with matching owners have been distributed to honest parties. However, this is only an issue early on, when only a single honest party might know an "important" element (the one holding the corresponding certificate) or the end stage, when already a supermajority of parties have obtained a certificate. Exploiting this, we add a step in which parties with certificates spread elements that contribute to certificates for the respective

values, without the need to hide from the adversary who these parties are.

DistributeCertificate

We describe the view of the protocol for party i. P_i inputs an array A_i of length n, where each index j holds an element owned by P_j. Initialize $A'_i := A_i, C_0 := C_1 := \perp$, $r := 1$. The protocol requires that $\lambda \ll n$, i.e., for a constant a we may choose, $\lambda \leq n/a$.

- While $r \leq R = O(\log^2(n)/\log(n/\lambda))$:
  - **Step 2r-1:**
    - While $C_0 = \perp$ or $C_1 = \perp$ and A_i contains a certificate C for a value v stored in neither C_0 or C_1: set C_b with the lowest b such that $C_b = \perp$ to C.
    - Invoke $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}_1}$ on input (A'_i, A_i) and denote the output as S_i. Set $A'_i := A_i$. For each $k = 1 \ldots n$, update $A_i[k] := A_i[k] \oplus S_i[k]$.
  - **Step 2r:** In parallel, for $b = 0$ *and* $b = 1$ do:
    - Let K_i be an array of n entries which (i) holds only $\perp$ elements if $C_b = \perp$ or (ii) the elements making up certificate $C_b \neq \perp$ (and $\perp$ elements for the remaining entries). If (i) applies, set $a_i = \perp$, otherwise set $a_i = v$, where v is the value C_b certifies.
    - Invoke an instance of $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}_2^{\vec{a}}}$ on input (A'_i, K_i) and denote the output as S_i. Here, $\vec{a}$ denotes the vector of values $(a_j)_{j \in \mathcal{H}}$ and $\mathcal{H}$ denotes the set of honest parties. (We stress that P_i only knows a_i.)
    - For each $k = 1 \ldots n$, update $A_i[k] := A_i[k] \oplus S_i[k]$.
- **Step** $O(\log^2(n)/\log(n/\lambda))$**:**
  - While $C_0 = \perp$ or $C_1 = \perp$ and A_i contains a certificate C for a value v stored in neither C_0 or C_1: set C_b with the lowest b such that $C_b = \perp$ to C.
- **Output Determination:** Output C_0 and C_1.

Fig. 5. The DistributeCertificate procedure

We now discuss how DistributeCertificate implements these ideas. It makes repeated calls to $\mathcal{F}_{\mathsf{pull}}$, which are parameterized with predicates $\mathsf{OrderRel}_1$ and $\mathsf{OrderRel}_2^{\vec{a}}$, induced, respectively, by Order_1 and Order_2. More precisely, $\mathsf{OrderRel}_1$ (and similarly, $\mathsf{OrderRel}_2^{\vec{a}}$) takes as input two elements, samples uniform, independent random coins, and executes $\mathsf{OrderIn}_1$ on behalf of the first party, P_j, using as input the first element from its input. The output of this invocation of $\mathsf{OrderIn}_1$ is then used to invoke $\mathsf{OrderOut}_1$ on behalf of the second party, P_i. The output of $\mathsf{OrderRel}_1$ is whatever $\mathsf{OrderOut}_1$ outputs. $\mathsf{OrderRel}_2^{\vec{a}}$ is parametrized by a vector $\vec{a}$ of values the honest parties seek to advertise,[7] i.e., party P_i wishes

[7] Note that these are implicitly leaked to the adversary as part of the description of $\mathcal{F}_{\mathsf{pull}}$ in respective instances.

to spread a certificate for value a_i. For technical reasons, we sometimes make the random coins used to call these predicates explicit by denoting them as ρ_1, ρ_2 (see the definition below).

Definition 7 (Predicates $\mathsf{OrderRel}_1$ and $\mathsf{OrderRel}_2^{\vec{a}}$). *Let e, e' be elements and let $\vec{a}$ be a vector of $n - t$ of elements. We define randomized predicates as:*

$$\mathsf{OrderRel}_1(e, e'; \rho_1, \rho_2) := (\mathsf{OrderOut}_1(e', \mathsf{OrderIn}(e; \rho_1); \rho_2)$$

and $$\mathsf{OrderRel}_2^{\vec{a}}(e, e'; \rho_1, \rho_2) := (\mathsf{OrderOut}_2^{\vec{a}}(e', \mathsf{OrderIn}(e; \rho_1); \rho_2).$$

We remark that it is possible that a party does not have such a certificate to spread. In this case, it sets K_i to n empty elements when calling $\mathcal{F}_{\mathsf{pull}}$ with $\mathsf{OrderRel}_2^{\vec{a}}$, and thus does not send an element for any query. Hence, in its role as a sender, its only purpose is to provide cover traffic for the other parties.

We now describe the protocol in more detail. At party P_i, $\mathsf{DistributeCertificate}$ initializes the lists A_i and A_i' to the list of elements it holds and then performs R iterations of the following. Unless it already has two certificates C_0 and C_1 stored, it checks whether A_i contains a new element. If it does, P_i stores it in C_0 or C_1, respectively, and checks again. After this local computation, it invokes $\mathcal{F}_{\mathsf{pull}}$ with $\mathsf{OrderRel}_1$, where $K_i = A_i$ and $M_i = A_i'$. It then updates A_i in accordance with the new elements it learned from that call, but only after updating $A_i' := A_i$. As explained above, this realizes a deferred update of A_i', which lags behind by one iteration in terms of the information P_i holds. Subsequently, in two parallel calls to $\mathcal{F}_{\mathsf{pull}}$, parties seek to spread elements of their certificates. Accordingly, $\mathsf{OrderRel}_2^{\vec{a}}$ is used in these calls, where parties use their first and second advertised value for their entry a_i of $\vec{a}$, respectively. Note that both calls use $M_i = A_i'$ and K_i is set to the respective certificate (or a list of $\perp$ entries, if P_i has no value to advertise for this instance). Thus, no information about the new elements that P_i learned in the previous step is leaked. Once the R loop iterations are complete, P_i checks again for new certificates and then outputs the up to two certificates it holds.

The detailed analysis of $\mathsf{DistributeCertificate}$ is deferred to Appendix D.

4 Binary Consensus and Parallel Broadcast

In this section, we present our main protocols for binary consensus and parallel broadcast. We begin by showing a binary consensus protocol that is used as a building block for parallel broadcast.

4.1 Binary Consensus

Our protocol for binary consensus is nearly identical to that of [28], and differs in that it uses a graded consensus protocol that outputs values with 3 possible grades. In the following, we refer to disjoint subsets of K_1 and K_2 such that $K = K_1 \cup K_2$ as *balanced* if their sizes differ by at most 1.

In more detail, our approach uses a subroutine $\mathsf{GradedConsensus}$ (described in the next section) in order to efficiently implement binary consensus based

on recursion. The idea is to maintain an opinion at each party initialized to its input. If the opinions of all honest parties agree, they should not change; if they do not agree, we use recursive calls on half of the parties and spread the outcome to the rest of the parties.

Binary Consensus

We describe the view of party P_i with input $x_i \in \{0,1\}$, which initializes the variable $v_i := x_i$. We denote by K the set of parties that are participating in the protocol. (Initially, $K := P$). Set $j := 1$

- **If** $|K| = 1$ **then** return v_i. **Else** Split K into two subsets K_1 and K_2 of balanced size.
  For $j = 1, 2$:
  - **Step** $2j-1$**:** Invoke GradedConsensus with input v_i among the parties in K and let (v, g) be the output. If $g \geq 1$, set $v_i = v$
  - **Step** $2j$**: If** $P_i \in K_j$ **then** recursively invoke BinaryConsensus with inputs v_i among parties in K_j and send the output v to all parties in K; **else** wait until the step finishes.
    If $P_i \in K$ receives the same value v from a majority of parties in K_j and $g \leq 1$ **then** set $v_i = v$
- **Output Determination:** Return v_i.

Fig. 6. The BinaryConsensus Binary Consensus Protocol.

To ensure agreement and validity, we invoke GradedConsensus on the opinions of the current subset of parties. Parties with confidence level 2 use the output value as their new opinion. We then split the subset into two halves, and call the Binary Consensus recursively on each half (sequentially). The output of each such invocation is spread to all parties of the original subset. Parties that did not have grade 2, may now adopt the majority opinion output of the recursive call (or default to their opinion in case there is no majority or the output is $\perp$). It follows that

- If all inputs for GradedConsensus agree, no party changes opinion.
- If *some* honest party has grade 2 and the recursive call has a majority of honest parties: the agreement of GradedConsensus ensures that all honest parties of the recursive call input the same value, and by the validity of BinaryConsensus they end up in agreement, which spread to all the honest parties, since this is the majority opinion in the recursive step.
- If no honest party has grade 2 and the recursive call has a majority of honest parties: the agreement property of BinaryConsensus ensures agreement in that subset, which is then adopted by all honest parties in the original set, since this is the majority opinion.

The correctness then follows because at least one subset must have an honest majority. In Appendix E we formally prove the correctness of BinaryConsensus given in Lemma 2.

Lemma 2. BinaryConsensus *(Fig. 6) implements Binary Consensus.*

The communication complexity of the protocol is bounded in Lemma 3, whose proof also appears in Appendix E. Note that for $\lambda = n^{1-\Omega(1)}$ the communication becomes $O(n^2\kappa\log(n))$.

Lemma 3. BinaryConsensus *has a communication complexity of* $O(n\lambda\kappa\log^3(n) + n^2\kappa\log^2(n)/\max\{\log(n/\lambda), 1\})$.

Theorem 1 now follows directly from Lemma 2 and Lemma 3.

4.2 Graded Consensus

In this section, we explain how to obtain an efficient routine for multivalued graded consensus from our routine DistributeCertificate described in the previous section. As we will now discuss, DistributeCertificate readily enables us to efficiently implement graded consensus for inputs of length $O(\kappa)$. Our multivalued graded consensus protocol, GradedConsensus, is depicted in Figure 7. Informally, the protocol performs the following steps.

1. Each party multicasts its signed input. If a party can form a certificate for some value v from the received signatures, it does so.
2. We invoke DistributeCertificate in order to spread this certificate efficiently (if it was assembled in the first step)
3. We invoke DistributeCertificate again, using at each party up to two certificates that were assembled or received so far (on differing values) as input.

The output is then determined as follows: (i) If a certificate for some v was obtained in Step 1 and no other is obtained later, output $(v, 2)$. (ii) If a certificate for some v was obtained in Step 1 and another later, output $(v, 1)$. (iii) If a single certificate for some v was obtained in Steps 2 or 3, but none before, output $(v, 1)$. (iv) In all other cases, output $(\perp, 0)$.

It is easy to verify graded validity from Property (i) of Theorem 3 and graded agreement from Property (ii), cf. Lemma 4. The communication complexity of the protocol is dominated by that of DistributeCertificate, cf. Lemma 5.

In Appendix E we prove the following lemmas that certify the correctness of our GradedConsensus protocol, and analyze its communication.

Lemma 4. GradedConsensus *(Fig. 7) implements Graded Consensus.*

Lemma 5. GradedConsensus *has a communication complexity of* $O(2\kappa n^2 + DC)$*, where* DC *is the communication complexity of an invocation of* DistributeCertificate.

4.3 Collective Broadcast

As a next stepping stone on our way to achieving parallel broadcast, we now give our algorithm for collective broadcast (Definition 5), depicted in Figure 8. This construction utilizes a cryptographic accumulator (Definition 10) and a linear erasure correcting code (Definition 9). Specifically, we utilize a $(t+1, n)$-Reed Solomon code capable of decoding any codeword corrupted by at most

Multivalued Graded Consensus

We describe the view of party P_i. Let $x \in \{0,1\}^\kappa$ denote P_i's input. Initially set $v = \perp$, $g = 0$, $\mathcal{C} = \emptyset$, and A_C to an empty array of length n.

- **Step 1:** Send $(x, \langle x \rangle_i)$ to all parties.
- **Step 2:** If P_i receives multiple distinct and valid signed values from P_j, it uses any two signatures to form a double element (owned by P_j), and discards the rest of the signatures owned by P_j, if any. For any value $\hat{x} \in \{0,1\}^*$ for which P_i has received $t+1$ double elements or single elements with $\hat{x}$ as the payload, P_i combines these elements into a certificate $C_{\hat{x}}$. P_i stores all certificates assembled in this manner in a list $\mathcal{C}$. **If** $\mathcal{C} = \{C_{\hat{x}}\}$, **then** P_i sets $v = \hat{x}$ and $g = 2$. For all $j \in [n]$ from whom P_i received a single or a double element in this step, P_i stores that element in $A_C[j]$. Invoke DistributeCertificate on input A_C.
- **Step 3: If** DistributeCertificate outputs certificates for multiple unequal values or output a certificate for a value v' s.t. $\perp \neq v' \neq v \neq \perp$ **then** P_i sets $v = \perp$ and $g = 0$; **else if** DistributeCertificate outputs a single certificate $C_{v'}$ for a value v' and $\mathcal{C} = \emptyset$ then set $v = v'$, $g = 1$ and set $\mathcal{C} = \{C_{v'}\}$. For all $j \in [n]$ s.t. P_i received a single or a double element with P_j as the owner in this step (directly or indirectly), P_i stores that element in $A_C[j]$. Invoke DistributeCertificate on input A_C.
- **Output Determination: If** DistributeCertificate outputs a single certificate $C_{v'}$ for a value v' and $C = \emptyset$ **then** set $v = v'$ and $g = 1$; **else if** DistributeCertificate output certificates for multiple unequal values or a certificate for a value v' s.t. $\perp \neq v' \neq v \neq \perp$ and $g = 2$ **then** set $g = 1$. Return (v, g).

Fig. 7. The GradedConsensus Multivalued Graded Consensus Protocol.

t erasures. We match the codes parameters to the length of the encoded message, so that the encoding of a message of length ℓ yields n symbols of length $O(\max\{\log(n), \ell/n\})$.

Recall that in the collective broadcast problem, a subset $S \subset K$ of the parties start each with a message m_i of length ℓ. The goal is that all K parties agree on an output message, where if all m_i were the same, this must be the output. Our collective broadcast implementation leverages GradedConsensus and BinaryConsensus in standard extension techniques.

- Each party in S encodes the message m via an error correction code into a codeword of n symbols $(cw_1, \ldots, cw_n)$. Using a cryptographic accumulator, the party computes a digest value acc on the n symbols, and for each symbol cw_i, it generates a witness wit_i. Then, each sender sends each $P_i \in K$ the tuple (cw_i, acc, wit_i).
- Each P_i receives a tuple (cw_i, acc, wit_i) from each sender; if the majority tuple is valid (i.e., it passes the accumulator's Verify), party P_i echos this

Collective Broadcast

We describe the view of party P_i with a set of participants K, a fixed sender set $S \subset K$, and input m_i. In the following, $t = \lfloor (|K| - 1)/2 \rfloor$.

- **Step 1: If** $P_i \in S$ **then** compute a list of $|K|$ codewords symbols $cw_1, \ldots, cw_{|K|}$ and an accumulator on the codeword as: $cw_1, \ldots, cw_{|K|} \leftarrow \mathsf{Encode}(m_i)$ and $acc \leftarrow \mathsf{Eval}(cw_1, \ldots, cw_{|K|}, ak)$. Send to each party $P_j \in K$ a message containing their symbol cw_j, acc, and their witness computed as $\mathsf{CreateWit}(ak, acc, cw_j)$.
- **Step 2:** Let cw_i and acc be the symbol and accumulator received in a majority of the messages. **If** there was no majority or no wit_i was received s.t. $\mathsf{Verify}(ak, acc, cw_i, wit_i)=1$ **then** invoke $\mathsf{GradedConsensus}$ on input 0; **else if** $\mathsf{Verify}(ak, acc, cw_i, wit_i)=1$ for some wit_i that was received **then** send a message containing cw_i, acc, and wit_i to all parties and invoke $\mathsf{GradedConsensus}$ on input acc. Let (acc^*, g) be the output of $\mathsf{GradedConsensus}$.
- **Step 3: If** $g = 2$ and P_i has received a set of at least $t + 1$ distinct codeword-symbols and tuple pairs (cw_j, wit_j) s.t. $\mathsf{Verify}(ak, acc^*, cw_j, wit_j)=1$ and P_i can decode m^* with $t + 1$ such symbols (and t blanks) as $m^* = \mathsf{Decode}(cw_i, \ldots, cw_{|K|})$ **then** invoke $\mathsf{BinaryConsensus}$ with input 1; **else** invoke $\mathsf{BinaryConsensus}$ with input 0. Let x denote the output.
- **Step 4: If** $x = 0$ **then** return 0^n; **else if** P_i has at least $t + 1$ codeword-symbols and witness pairs (cw_j, wit_j) for acc^* s.t. $\mathsf{Verify}(ak, acc^*, cw_j, wit_j)=1$ then decode m^* as $m^* = \mathsf{Decode}(cw_1, \ldots, cw_{|K|})$ (as in the previous step), (re-)compute the $|K|$ codeword symbols $cw_1, \ldots, cw_{|K|} \leftarrow \mathsf{Encode}(m^*)$ and $|K|$ corresponding witnesses $wit_j \leftarrow \mathsf{CreateWit}(ak, acc, cw_j)$, and send a message to each party P_j with their symbol cw_j and witness wit_j.
- **Step 5: If** P_i received a symbol cw_i and witness wit_i such that $\mathsf{Verify}(ak, acc^*, cw_i, wit_i) = 1$ **then** send a message to each party containing cw_i and wit_i.
- **Output Determination:** Once from $t + 1$ distinct parties P_j, cw_j and wit_j such that $\mathsf{Verify}(ak, acc^*, cw_j, wit_j) = 1$ are received, decode from them the message $m^* = \mathsf{Decode}(cw_i, \ldots, cw_{|K|})$ (as in prior steps). Return m^*.

Fig. 8. The $\mathsf{CollectiveBC}$ Collective Broadcast Protocol.

tuple to all other parties.

- The parties invoke $\mathsf{GradedConsensus}$ on the accumulator digest value acc, where each party uses the majority value it has received (if valid), or defaults to 0, otherwise.
- Next, the parties invoke $\mathsf{BinaryConsensus}$, where each party uses input 1 only if (i) $\mathsf{GradedConsensus}$ resulted in confidence 2 and (ii) the party can decode the message based on the received codeword symbol; Namely, it has

at least $t+1$ different cw_i's that are all valid (verified by a witness) for the accumulator output by GradedConsensus.

– If BinaryConsensus outputs 0, all parties output 0. If it outputs 1, do:
  - If a party that can decode the message for the accumulator given by the output of GradedConsensus, it (re-)computes all codeword symbols and witnesses and sends each pair to the respective party.
  - Each party verifies its own symbol cw_i for the accumulator output by GradedConsensus and echos it with its witness to all other parties.
  - All parties decode the message from the valid received cw_i's, and output the message.

Verifying validity of this approach is straightforward, cf. Lemma 27. To prove agreement, observe that if BinaryConsensus outputs 1 by its validity property, at least one honest party must output an accumulator with confidence 2 from the call to GradedConsensus *and* be able to decode the message. Thus, all parties use the same accumulator digest value and learn their own cw_i and wit_i from this party. Since they can distinguish the correct codeword-symbol from spurious ones sent by dishonest parties (via the accumulator's Verify), they echo only valid alongside its witness, resulting in all parties obtaining all symbol/witness pairs belonging to honest parties. This is sufficient to decode the message belonging to the accumulator coming out of GradedConsensus.

In Appendix E we formally prove the correctness of the CollectiveBC protocol and analyze its communication. Namely, we show the following.

Lemma 6. CollectiveBC *(Fig. 8) implements Collective Broadcast.*

The next Lemma 7 bounds the communication complexity of our collective broadcast routine. For n parties and $\lambda = n^{1-\Omega(1)}$ it is $O(n\ell + n^2\kappa\log(n))$, where ℓ is the length of the broadcast message. The first term comes from the distribution of codeword symbols and the second from calling GradedConsensus and BinaryConsensus.

Lemma 7. CollectiveBC *has a communication complexity of* $O(|K|(\ell+\lambda\kappa\log^3(n))+|K|^2\kappa\log^2(n)/\max\{\log(n/\lambda),1\})$.

Corollary 3. *For* $\lambda = n^{1-\Omega(1)}$*, the communication complexity of* CollectiveBC *is* $O(|K|\ell + |K|^2\kappa\log(n))$.

4.4 Parallel Broadcast

Having described all prior subroutines in detail, we now give an updated and simplified description of our final parallel broadcast routine Parallel BC (Fig. 9).

Each party P_i starts the protocol with an input message m_i and initializes V to an empty array of length n. Parties multicast their signed inputs, and each time a party receives a signed message from another party P_j, it stores the message and signature in $V[j]$. The parties then invoke RecursiveParallelBC (Fig. 10). As explained in the informal description in Section 1.1, in each recursive step, the protocol splits the set of parties into two equal sized subsets on which the

protocol is recursively invoked. Once the recursive calls on each of the subsets complete, each of the two subsets of parties has a list V. Subsequently, each of the two subsets of parties acts as the sender set in an invocation of our collective broadcast protocol CollectiveBC to broadcast their list V to all of the parties. Upon completion of both broadcasts, a party locally merges the two lists, dropping any excess messages, as these can be originated only by corrupted parties.

Parallel Broadcast

We describe the protocol from the view of party P_i. P_i has input a message $m_i \in \{0,1\}^\ell$. V is initialized to an array of length n with 0 in each index.

- **Step 1:** Send $(m_i, \langle m_i \rangle_i)$ to all.
- **Step 2: For** every message $(m_j, \langle m_j \rangle_j)$ received from $p_j \in P$: **If** $\langle m_j \rangle_j$ is a valid signature on m_j **then** set $V[j] = (m_j, \langle m_j \rangle_j)$. Run RecursiveParallelBC (among all the parties) on input V.
- **Output Determination:** When RecursiveParallelBC returns with output V^*, return V^*.

Fig. 9. The Parallel BC Parallel Broadcast Protocol.

Recursive Parallel Broadcast

We describe the protocol from the view of party P_i and let K denote the set of protocol participants. P_i has input array V of length n. Initialize V^* to an empty array of length n.

- **If** P_i is the only party in K **then** return V.
- **Else** split K into two balanced subsets K_0 and K_1. For $j \in \{0,1\}$ in parallel do
  1. **If** P_i is in K_j **then** invoke RecursiveParallelBC with input V among parties in K_j and let V' denote the output; and
  2. invoke CollectiveBC with input V' and sender set K_j, and let V_j denote the output.

  For each index k in V^*: **if** $V_1[k]$ contains a tuple consisting of a message m and a valid signature on m from P_j **then** set $V^*[k] = m$; **else if** $V_2[k]$ contains a tuple consisting of a message m and a valid signature on m from P_j **then** set $V^*[k] = m$.
  Return V^*.

Fig. 10. The RecursiveParallelBC Recursive Parallel Broadcast Protocol.

In Appendix E we analyze the correctness and communication of Parallel BC. Theorem 2 will then follow from Lemmas 8 and 9.

Lemma 8. Parallel BC *(Fig. 9) implements Parallel Broadcast.*

Lemma 9. *The* Parallel BC *protocol incurs*

$$O\left(n^2\ell\log(n) + n\lambda\kappa\log^4(n) + n^2\kappa\cdot\frac{\log^2(n)}{\max\{\log(n/\lambda),1\}}\right)$$

communication complexity.

Corollary 4. *For* $\lambda = n^{1-\Omega(1)}$*, the* Parallel BC *protocol has communication complexity* $O(n^2(\ell+\kappa)\log(n))$*.*

Acknowledgments. R. Gelles is supported in part by the United States – Israel Binational Science Foundation (BSF), grant No. 2020277. He would like to thank CISPA Helmholtz Center for Information Security for hosting him while part of this research was conducted. This work is funded by the European Union, ERC-2023-STG, Project ID: 101116713. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

References

1. Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Asymptotically free broadcast in constant expected time via packed VSS. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 384–414. Springer, Heidelberg, November 2022.
2. Ittai Abraham, T.-H. Hubert Chan, Danny Dolev, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Communication complexity of byzantine agreement, revisited. In Peter Robinson and Faith Ellen, editors, *38th ACM PODC*, pages 317–326. ACM, July / August 2019.
3. Ittai Abraham, Kartik Nayak, and Nibesh Shrestha. Communication and round efficient parallel broadcast protocols. *IACR Cryptol. ePrint Arch.*, page 1172, 2023.
4. Gilad Asharov and Anirudh Chandramouli. Perfect (parallel) broadcast in constant expected rounds via statistical VSS. In *EUROCRYPT (to appear)*, page 376, 2024.
5. Piotr Berman, Juan Garay, and Kenneth J. Perry. Bit optimal distributed consensus. *Computer Science*, pages 313–321, 1992.
6. Erica Blum, Jonathan Katz, Chen-Da Liu-Zhang, and Julian Loss. Asynchronous byzantine agreement with subquadratic communication. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 353–380. Springer, Heidelberg, November 2020.
7. Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of CRYPTOLOGY*, 13:143–202, 2000.
8. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.

9. Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In Margo I. Seltzer and Paul J. Leach, editors, *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI), New Orleans, Louisiana, USA, February 22-25, 1999*, pages 173–186. USENIX Association, 1999.
10. T.-H. Hubert Chan, Rafael Pass, and Elaine Shi. Sublinear-round byzantine agreement under corrupt majority. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 246–265. Springer, Heidelberg, May 2020.
11. Pierre Civit, Muhammad Ayaz Dzulfikar, Seth Gilbert, Rachid Guerraoui, Jovan Komatovic, and Manuel Vidigueira. DARE to agree: Byzantine agreement with optimal resilience and adaptive communication. In *ACM PODC*. ACM, 2024.
12. Brian A. Coan and Jennifer L. Welch. Modular construction of a byzantine agreement protocol with optimal message bit complexity. *Inf. Comput.*, 97(1):61–85, 1992.
13. Daniel Collins, Sisi Duan, Julian Loss, Charalampos Papamanthou, Giorgos Tsimos, and Haochen Wang. Towards optimal parallel broadcast under a dishonest majority. Cryptology ePrint Archive, Report 2024/974, 2024.
14. Danny Dolev and Rüdiger Reischuk. Bounds on information exchange for byzantine agreement. In Robert L. Probert, Michael J. Fischer, and Nicola Santoro, editors, *1st ACM PODC*, pages 132–140. ACM, August 1982.
15. Danny Dolev and H. Raymond Strong. Polynomial algorithms for multiple processor agreement. In *14th ACM STOC*, pages 401–407. ACM Press, May 1982.
16. Danny Dolev and Andrew Chi-Chih Yao. On the security of public key protocols (extended abstract). In *22nd FOCS*, pages 350–357. IEEE Computer Society Press, October 1981.
17. Paul Feldman and Silvio Micali. Optimal algorithms for byzantine agreement. In *20th ACM STOC*, pages 148–161. ACM Press, May 1988.
18. Matthias Fitzi and Jesper Buus Nielsen. On the number of synchronous rounds sufficient for authenticated byzantine agreement. In Idit Keidar, editor, *Distributed Computing, 23rd International Symposium, DISC 2009, Elche, Spain, September 23-25, 2009. Proceedings*, volume 5805 of *Lecture Notes in Computer Science*, pages 449–463. Springer, 2009.
19. Juan A. Garay, Jonathan Katz, Chiu-Yuen Koo, and Rafail Ostrovsky. Round complexity of authenticated broadcast with a dishonest majority. In *48th FOCS*, pages 658–668. IEEE Computer Society Press, October 2007.
20. Juan A. Garay and Yoram Moses. Fully polynomial byzantine agreement in t+1 rounds. In *25th ACM STOC*, pages 31–41. ACM Press, May 1993.
21. Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*, pages 51–68. ACM, 2017.
22. Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM J. Comput.*, 17(2):281–308, 1988.
23. Jonathan Katz and Chiu-Yuen Koo. On expected constant-round protocols for byzantine agreement. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 445–462. Springer, Heidelberg, August 2006.
24. Valerie King and Jared Saia. Breaking the $O(n^2)$ bit barrier: scalable byzantine agreement with an adaptive adversary. In Andréa W. Richa and Rachid Guerraoui, editors, *29th ACM PODC*, pages 420–429. ACM, July 2010.

25. Valerie King, Jared Saia, Vishal Sanwalani, and Erik Vee. Scalable leader election. In *17th SODA*, pages 990–999. ACM-SIAM, January 2006.
26. Eyal Kushilevitz and Noam Nisan. *Communication Complexity*. Cambridge University Press, 1996.
27. Silvio Micali. Very simple and efficient byzantine agreement. In Christos H. Papadimitriou, editor, *ITCS 2017*, volume 4266, pages 6:1–6:1, 67, January 2017. LIPIcs.
28. Atsuki Momose and Ling Ren. Optimal communication complexity of authenticated byzantine agreement. *arXiv preprint arXiv:2007.13175*, 2020.
29. Atsuki Momose and Ling Ren. Optimal communication complexity of authenticated byzantine agreement. In Seth Gilbert, editor, *35th International Symposium on Distributed Computing, DISC 2021, October 4-8, 2021, Freiburg, Germany (Virtual Conference)*, volume 209 of *LIPIcs*, pages 32:1–32:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.
30. Marshall C. Pease, Robert E. Shostak, and Leslie Lamport. Reaching agreement in the presence of faults. *J. ACM*, 27(2):228–234, 1980.
31. Irving S Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the society for industrial and applied mathematics*, 8(2):300–304, 1960.
32. Shravan Srinivasan, Julian Loss, Giulio Malavolta, Kartik Nayak, Charalampos Papamanthou, and Sri Aravinda Krishnan Thyagarajan. Transparent batchable time-lock puzzles and applications to byzantine consensus. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 554–584. Springer, Heidelberg, May 2023.
33. Georgios Tsimos, Julian Loss, and Charalampos Papamanthou. Gossiping for communication-efficient broadcast. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 439–469. Springer, Heidelberg, August 2022.
34. David Wajc. Negative association: definition, properties, and applications. Manuscript, available from https://goo.gl/j2ekqM, 2017.
35. Jun Wan, Hanshen Xiao, Srinivas Devadas, and Elaine Shi. Round-efficient byzantine broadcast under strongly adaptive and majority corruptions. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 412–456. Springer, Heidelberg, November 2020.
36. Jun Wan, Hanshen Xiao, Elaine Shi, and Srinivas Devadas. Expected constant round byzantine broadcast under dishonest majority. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 381–411. Springer, Heidelberg, November 2020.

A Coding and Cryptographic Primitives

We introduce here several basic cryptographic and coding primitives utilized in our constructions.

Definition 8 (Public Key Encryption (PKE)). *A public key encryption scheme is a tuple of PPT algorithms (*KeyGen*,* ENC*,* DEC*) such that:*

- KeyGen *is a randomized* key generation algorithm *that takes as input the security parameter κ and outputs a public-secret key pair (pk, sk).*
- ENC *is a randomized* encryption algorithm *that takes as input a public key pk and a message m and outputs a ciphertext c.*

– DEC *is a deterministic* decryption algorithm *that takes as input a decryption key sk and a ciphertext c and outputs a message m, where possibly $m = \perp$ in case of failure.*

We assume a PKE scheme with perfect correctness, i.e., for any $m \in \{0,1\}^*$, and for any key pair $(pk, sk) \leftarrow \mathsf{KeyGen}$, $\mathsf{DEC}(sk, \mathsf{ENC}(pk, m)) = m$. We recall the standard CPA-security definition for PKE in the supplementary material.

Definition 9 (Linear Erasure Codes). *We use standard (n, k) Reed-Solomon codes. A Reed-Solomon (RS) code [31] is a linear error correction code over the finite field $\mathbb{F}_{2^a}$, parameterized by n and k with $n \leq 2^a - 1$, given by the tuple of deterministic algorithms (*Encode*, *Decode*) such that:*

– $\mathsf{Encode} : \mathbb{F}_{2^a}^k \to \mathbb{F}_{2^a}^n$ *is a deterministic encoding algorithm that takes as input k data symbols $(m_1, \ldots, m_k) \in \mathbb{F}_{2^a}^k$ and outputs a codeword $(s_1, \ldots, s_n) \in \mathbb{F}_{2^a}^n$ of length n.*

– $\mathsf{Decode} : \mathbb{F}_{2^a}^n \to \mathbb{F}_{2^a}^k$ *is a deterministic decoding algorithm that takes as input a (possibly corrupted) codeword $(s'_1, \ldots, s'_n)$ of length n and outputs k symbols. The decoding can tolerate up to f substitution errors and e erasures given that $2f + e \leq n - k$; that is, if $(s'_1, \ldots, s'_n)$ can be obtained by corrupting (altering) f symbols and erasing e symbols of some $(s_1, \ldots, s_n) = \mathsf{Encode}(m_1, \ldots, m_k)$, then $(m_1, \ldots, m_k) \leftarrow \mathsf{Decode}(s'_1, \ldots, s'_n)$.*

In this work, we use Reed-Solomon codes with $k = t + 1$. For our purposes, code word symbols resulting from encodings of messages of length ℓ will be of size $\max\{\log(n), \ell/n\}$.

Definition 10 (Cryptographic Accumulator). *A cryptographic accumulator is a tuple of PPT algorithms (*Gen*, *Eval*, *$\mathsf{CreateWit}$*, *Verify*) such that:*

– Gen *is a randomized* accumulator key generation algorithm *that takes in the security parameter κ and a number $m \in \mathbb{N}$ and outputs an accumulator key ak. We assume that ak implicitly contains m.*

– Eval *is a deterministic* accumulation algorithm *that takes in an accumulator key ak for some size m, a set to be accumulated $D = \{d_1, \ldots, d_{m'}\}$ such that $m' \leq m$ and outputs an accumulation value z for D.*

– $\mathsf{CreateWit}$ *is a randomized* witness creation algorithm *that takes as input an accumulator key ak, an accumulation value z for D, and a value d_i. If $d_i \notin D$, it outputs $\perp$; otherwise, it outputs a witness w_i.*

– Verify *is a deterministic* verification algorithm *that takes as input an accumulator key ak, an accumulation value z for D, a witness w_i, and a value d_i. It outputs 1 (accept) or 0 (reject). The function accepts if and only if $w_i \neq \perp$ was generated by* $\mathsf{CreateWit}$ *on d_i and the accumulated value z.*

We require that the cryptographic accumulator satisfies *collision freeness*, meaning informally that, for a given key ak, set D, and accumulator value z, one cannot generate witnesses for elements that are not in the set. Since we are restricted to constructions that do not rely on trusted setup, our accumulators will be of size κ whereas witnesses will be of size $\kappa \cdot \log(m)$ (when accumulating

values from a set of size m). This can be achieved using Merkle trees, which can be instantiated without a trusted dealer in the random oracle model. Much like signatures, we follow the common convention of treating hash-based primitives as idealized.

B Equality Testing

We will use the following interactive two-party protocol for equality testing of two strings, with one-sided error and a single round of communication. See e.g., [26].

Definition 11 (Randomized Two-Party Single-Round Equality Testing with One-sided Error). *In the* randomized two-player equality testing *problem, Alice and Bob are each given N-bit strings x and y, respectively, and the goal is for Bob to determine whether or not $x = y$, with probability of error at most ε. The error probability is one-sided: when $x = y$ Bob must indicate so with probability 1.*

The protocol may be randomized—each of the parties has access to a (sufficiently long) string of independent, unbiased random bits.

The following theorem, while not written explicitly in [26], is a well known folklore extension of the equality testing protocol given in [26]. The theorem states that randomized equality on bit strings of length N can be solved with one-sided error probability ϵ by communicating $O(\log(N/\epsilon))$ bits from Alice to Bob.

Theorem 4. *There exists a two-party protocol* $(\mathsf{EqIn}, \mathsf{EqOut})$ *with the following properties. Alice runs* EqIn *with input $x \in \{0,1\}^N$ and uniformly random coins $r_A \in \{0,1\}^{O(\log(N/\epsilon))}$ and sends a single message m of length $O(\log(N) + \log(\varepsilon^{-1}))$ to Bob. Bob runs* EqOut *on input $y \in \{0,1\}^N$ and m and outputs a bit b such that (1) $x = y$ implies $b = 1$ and (2) $x \neq y$ implies $b = 0$ with probability at least $1 - \epsilon$ over the choice of r_A.*

We use this equality test on length-ℓ messages with error probability $\epsilon = 1/n$.

Corollary 5. *There exists a two-party protocol* $(\mathsf{EqIn}, \mathsf{EqOut})$ *with the following properties. Alice runs* EqIn *with input $x \in \{0,1\}^\ell$ and uniformly random coins $r_A \in \{0,1\}^{O(\log(\ell)+\log(n))}$ and sends a single message m of length $O(\log(\ell) + \log(n))$ to Bob. Bob runs* EqOut *on input $y \in \{0,1\}^N$ and m and outputs a bit b such that (1) $x = y$ implies $b = 1$ and (2) $x \neq y$ implies $b = 0$ with probability at least $1 - 1/n$ over the choice of r_A.*

C Pull Securely Realizes $\mathcal{F}_{\mathsf{pull}}$

We prove security of our protocols in the MPC framework [7].

MPC Model. One part of our parallel broadcast protocol is modeled using the ideal functionality $\mathcal{F}_{\mathsf{pull}}$ (see Figure 1). To show that a specific implementation realizes $\mathcal{F}_{\mathsf{pull}}$, we use the definition of synchronous MPC in the standalone model by Canetti [7] which we briefly recall here. Let f be a (possibly randomized) function that takes n inputs. In the real world execution of a protocol Π for computing f, party P_i initially holds 1^κ and an auxiliary input z. The goal of running Π is for all honest parties P_i to learn output y_i, where $[y_1, \ldots, y_n] \leftarrow f(x_1, \ldots, x_n)$. During the execution, the adversary $\mathcal{A}$ can corrupt any party adaptively, upon which it learns the internal state of this party. At the end of the execution, each honest party outputs its local output (as instructed by Π) and the adversary $\mathcal{A}$ outputs its entire view. We write $\mathbf{Real}_{\Pi,\mathcal{A}}(1^\kappa, \mathbf{x}, z)$ to denote the distribution over the vector of outputs of honest parties and the set of corrupted parties in the above (real-world) experiment. We define the security of Π relative to an ideal world where a trusted party securely computes f and outputs the result to all parties. Parties hold inputs as above. The ideal world adversary is denoted as $\mathcal{S}$. The ideal-world execution proceeds as follows.

- **Initial corruptions.** $\mathcal{S}$ adaptively corrupts parties and learns their inputs.
- **Evaluation by a trusted party.** The inputs $\mathbf{x}$ of honest parties are sent to the trusted party. The ideal-world adversary $\mathcal{S}$ can specify the inputs on behalf of any of the parties it has corrupted. The trusted party evaluates the function f on these inputs and auxiliary input z and returns each computed output y_i to the respective party P_i.
- **Additional corruptions.** At any point in time (after output has been provided by the trusted party), $\mathcal{S}$ may adaptively corrupt additional parties P_i.
- **Output.** Every honest party P_i outputs (x_i, y_i).
- **Post-execution corruptions.** $\mathcal{S}$ may corrupt additional parties and output (any function of) its view.

$\mathbf{Ideal}_{f,\mathcal{S}}(1^\kappa, \mathbf{x}, z)$ denotes the distribution over the vector of outputs of honest parties and the set of corrupted parties in the above (ideal-world) experiment.

Definition 12 (Secure Computation). *Π $t-$securely computes f if for all PPT adversaries $\mathcal{A}$ corrupting at most t parties, there exists a PPT simulator $\mathcal{S}$ such that*

$$\{\mathbf{Real}_{\Pi,\mathcal{A}}(1^\kappa, \boldsymbol{x}, z)\}_{k\in\mathbb{N},\boldsymbol{x},z\in\{0,1\}^*} \approx \{\mathbf{Ideal}_{f,\mathcal{S}}(1^\kappa, \boldsymbol{x}, z)\}_{k\in\mathbb{N},\boldsymbol{x},z\in\{0,1\}^*}.$$

Definition 13 (CPA Security). *Let $\mathsf{PKE} = (\mathsf{KeyGen}, \mathsf{ENC}, \mathsf{DEC})$ be a public key encryption scheme and consider the following experiment with an adversary $\mathcal{A}$ titled $\mathsf{CPA}^{\mathcal{A}}_{\mathsf{PKE},\mathsf{b}}$:*

1. *Run $\mathsf{KeyGen}(1^\kappa)$ to obtain keys (pk, sk).*
2. *Adversary $\mathcal{A}$ receives pk and outputs a pair of messages m_0, m_1.*
3. *Ciphertext $c \leftarrow \mathsf{ENC}_{pk}(m_b)$ is given to $\mathcal{A}$.*
4. *When $\mathcal{A}$ outputs a bit b' the experiment returns b'.*

We say that PKE *has* indistinguishability against a chosen plaintext attacks (IND-CPA) *if for all PPT adversaries* $\mathcal{A}$*, we have*

$$|\Pr[\mathsf{CPA}^{\mathcal{A}}_{\mathsf{PKE},1} = 1] - \Pr[\mathsf{CPA}^{\mathcal{A}}_{\mathsf{PKE},0} = 1]| \leq \mathsf{negl}(\kappa)$$

We now proceed to prove in Lemma 10 that Pull securely instantiate $\mathcal{F}_{\mathsf{pull}}$. As a remark on notation, we note that for ease of presentation, both Pull and $\mathcal{F}_{\mathsf{pull}}$ were presented from the view of a party P_i. Among other things, this allowed us to avoid indexing variables with i. However, as the proofs below deal with all parties at once, we now find it convenient to extend the indexing of these variables in compared as to how they are presented in Pull and $\mathcal{F}_{\mathsf{pull}}$. Our general convention will be that any variable indexed as $v_{j,i}$ that is sent, is sent by P_i and received by P_j. To this end, we rename $R_{\gamma,j}, \mathsf{Ord}_{\gamma,j}$ from their naming convention in Pull to $R_{\gamma,j,i}, \mathsf{Ord}_{\gamma,j,i}$ and rename $pk_{1,j}, pk_{2,j}$ to $pk_{1,j,i}, pk_{2,j,i}$, respectively.

Lemma 10. *Let* $\mathsf{PKE} = (\mathsf{KeyGen}, \mathsf{ENC}, \mathsf{DEC})$ *denote an IND-CPA PKE scheme and assume secure erasures. Then, the following statements are true:*

- $\mathsf{Pull}^{\mathsf{Order}_1}$ *t-securely computes* $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}_1}$
- *for all vectors* $\vec{a} \in (\{0,1\}^\kappa)^{n-t}$*,* $\mathsf{Pull}^{\mathsf{Order}^{\vec{a}}_2}$ *t-securely computes* $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}^{\vec{a}}_2}$*.*

Proof (Proof of Lemma 10). We show the proof for $\mathsf{Pull}^{\mathsf{Order}_1}$ and $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}_1}$; the proof for $\mathsf{Pull}^{\mathsf{Order}^{\vec{a}}_2}$ and $\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}^{\vec{a}}_2}$; proceeds analogously (for all $\vec{a}$). We start by describing a simulator $\mathcal{S}$.

- **Transcript simulation for** P_i**.** To simulate an honest party P_i in protocol $\mathsf{Pull}^{\mathsf{Order}_1}$ with input M_i, K_i, $\mathcal{S}$ proceeds, for each $j \in [n]$, as follows.
  - **Step One.** To simulate Step One of $\mathsf{Pull}^{\mathsf{Order}_1}$, it generates two pairs $(sk_{1,j,i}, pk_{1,j,i}) \leftarrow \mathsf{KeyGen}(1^\kappa)$ and $(sk_{2,j,i}, pk_{2,j,i}) \leftarrow \mathsf{KeyGen}(1^\kappa)$ and sends both public keys $pk_{1,j,i}, pk_{2,j,i}$ to P_j as a message of the form $\langle \textsc{ReportKey}, pk_{1,j,i}, pk_{2,j,i} \rangle$. Next, $\mathcal{S}$ samples lists $R_{\gamma,i,j}$ and $\mathsf{Ord}_{\gamma,i,j}$ for all $\gamma \in [c]$ as follows.
    - Set $R_{\gamma,j,i}$ to be a uniform permutation of $[n]$, truncated to length $\lceil \lambda / \log(n) \rceil$ (padded with 0 if $|M_i| < \lceil \lambda / \log(n) \rceil$).
    - For each $k \in [\lceil \lambda / \log(n) \rceil]$, set $\mathsf{Ord}_{\gamma,j,i}[k] = \mathsf{OrderIn}_1(M_i[R_{\gamma,j,i}[k]])$.

    These are the simulated lists P_i will send to P_j. For each P_j, $\mathcal{S}$ also generates lists that are sent from P_j to P_i in the simulation in a similar fashion. Namely, for all $\gamma \in [c]$, it does as follows.
    - Set $R_{\gamma,i,j}$ to be a uniform permutation of $[n]$, truncated to length $\lceil \lambda / \log(n) \rceil$ (padded with 0 if $|M_j| < \lceil \lambda / \log(n) \rceil$).
    - For each $k \in [\lceil \lambda / \log(n) \rceil]$, set $\mathsf{Ord}_{\gamma,i,j}[k] = \mathsf{OrderIn}_1(M_j[R_{\gamma,i,j}[k]])$.

  For the remaining steps of the protocol, $\mathcal{S}$ now uses these lists and keys as follows to simulate the messages that P_i is instructed to send to P_j in $\mathsf{Pull}^{\mathsf{Order}_1}$. Note that we write that P_i receives a message from P_j, we mean that this message has was either sent by a corrupted party P_j or from $\mathcal{S}$ itself, on behalf of an honest party P_j.

- **Step Two:** If $\langle \text{REPORTKEY}, pk_{1,i,j}, pk_{2,i,j} \rangle$ was sent to P_i from P_j in Step One of the simulation of $\mathsf{Pull}^{\mathsf{Order}_1}$, $\mathcal{S}$ sets, for all $\gamma \in [c]$, $Reqs[\gamma][j] = \mathsf{ENC}(pk_{1,i,j}, (R_{\gamma,j,i}, \mathsf{Ord}_{\gamma,j,i}))$. It sends $\langle \text{REQUEST}, Reqs[1][j], \ldots, Reqs[c][j] \rangle$ to P_j.
- **Step Three:** If the message $\langle \text{REQUEST}, \ldots \rangle$ was received from P_j in Step 2, decrypt it using $sk_{1,j,i}$ and let $(R_{\gamma,i,j}, \mathsf{Ord}_{\gamma,i,j}), \gamma \in [c]$ denote the c resulting decryptions. (If any of these decryptions fail, do nothing instead). For each $\gamma \in [c]$, create a corresponding $Resp_i[j][\gamma]$ as follows. Find the minimum k (if it exists) such that

$$\mathsf{OrderOut}(K_i[R_{\gamma,i,j}[k]], \mathsf{Ord}_{\gamma,i,j}[k]) = 1$$

  and set $Resp_i[j][\gamma] = \mathsf{ENC}(pk_{2,i,j}, K_i[R_{\gamma,i,j}[k]])$. Otherwise (i.e., if such k does not exist), set the corresponding $Resp_i[j][\gamma] = \mathsf{ENC}(pk_{2,i,j}, \text{NOMSG})$. $\mathcal{S}$ sends $\langle \text{RESPONSE}, Resp_i[j][1], \ldots, Resp_i[j][1] \rangle$ to P_j.
- **Simulating adversarial messages**: If the adversary sends a message $c = \mathsf{ENC}(pk_{2,j,i}, e)$ to P_i in this step via a corrupted party P_j, where element e is owned by P_k then $\mathcal{S}$ inputs $(\text{RESPONDDIRECT}, e, k, i)$ to $\mathcal{F}_{\mathsf{pull}}$.

– **Simulating Corruptions.** To simulate the corruption of a party P_i, $\mathcal{S}$ carries out the following steps upon handing over control of P_i to the adversary, depending on the point in time when P_i is corrupted. We implicitly assume that it always outputs P_i's input M_i, K_i.
  - **Static corruptions:** The simulator simply hands over control of the corrupted party to the adversary.
  - **Adaptive corruptions:** A party can be adaptively corrupted during four periods of the simulation. We describe the strategy of the simulator for each one:
    * **After sending in Step One, before sending in Step Two**: in this case, $\mathcal{S}$ outputs the secret keys $sk_{1,j,i}, sk_{2,j,i}$ for all j.
    * **After sending in Step Two, before sending in Step Three**: Due to the erasures applied in $\mathsf{Pull}^{\mathsf{Order}_1}$ in Step Two, $\mathcal{S}$ need only output $sk_{1,j,i}, sk_{2,j,i}$ and any ciphertexts that it generated for the simulation of Step Two of P_i's transcript.
    * **After sending in Step Three, before outputting**: Note that in $\mathsf{Pull}^{\mathsf{Order}_1}$, P_i erases $sk_{1,j,i}$ for all j and any plaintexts that it decrypted using these keys. Hence, $\mathcal{S}$ needs to only output $sk_{2,j,i}$ for all j.
    * **After outputting**: Since P_i erases its remaining secret keys $sk_{2,j,i}$ along with any plaintexts decrypted using these keys prior to outputting, $\mathcal{S}$ does not have to simulate anything further for this step.

The crucial property that $\mathcal{S}$ must ensure is that all the ciphertexts that are exchanged on behalf of simulated parties P_i over the course of simulated protocol transcript match what is output by honest parties by $\mathcal{F}_{\mathsf{pull}}$. We show that for any PPT environment $\mathcal{Z}$,

$$\{\mathbf{Real}_{\mathsf{Pull}^{\mathsf{Order}_1},\mathcal{A}}(1^\kappa, \mathbf{x}, z)\}_{k\in\mathbb{N},\mathbf{x},z\in\{0,1\}^*} \approx \{\mathbf{Ideal}_{\mathcal{F}_{\mathsf{pull}}^{\mathsf{OrderRel}_1},\mathcal{S}}(1^\kappa, \mathbf{x}, z)\}_{k\in\mathbb{N},\mathbf{x},z\in\{0,1\}^*}.$$

As we will show, this statement follows from the CPA security of the public key encryption scheme, as we show in more detail in the following. Our argument closely follows along the lines of Tsimos et al. [33]. The strategy of the proof is to consider a sequence of $2n^2$ hybrid experiments that starts with the sequence

$$H_0, H^1_{1,2}, H^1_{2,1}, H^2_{1,2}, H^2_{2,1}, ..., H^1_{1,n}, H^1_{n,1}, H^2_{1,n}, H^2_{n,1}$$

and continues with the sequence

$$H^1_{2,3}, H^1_{3,2}, H^2_{2,3}, H^2_{3,2}, ..., H^1_{n-1,n}, H^1_{n,n-1}, H^2_{n-1,n}, H^2_{n,n-1}.$$

H_0 corresponds to the ideal world execution with simulator $\mathcal{S}$ and the final experiment $H_{n,n-1}$ in the sequence corresponds to the real-world execution of protocol Pull. Importantly, the final experient $H_{n,n-1}$ still outputs the messages via $\mathcal{F}_{\mathsf{pull}}$, but the messages sent in between parties match these outputs. Loosely speaking, in every intermediate hybrid step $H_{i,j}$, the messages that are sent from P_i to P_j are altered. Instead of sending what the $\mathcal{S}$ would send, the experiment uses the values that are internally generated by $\mathcal{F}_{\mathsf{pull}}$ as the values from which the protocol messages, i.e., Order_1 messages and ciphertexts result. In the actual simulation, there will actually be $2n^2$ such hybrids, as we introduce further intermediary steps to change one of the protocol messages sent by P_i to P_j at a time. We write $H^1_{i,j}$ and $H^2_{i,j}$ to denote the hybrids corresponding to the steps where the first and the second message in the simulation from P_i to P_j are replaced, respectively. For technical reasons, we have to switch back and forth between switching the distributions of messages that are sent between parties.

As above, we rename some of the variables in $\mathcal{F}_{\mathsf{pull}}$ to make the distinction among different parties more clear for the proof. To this end, we rename $R'_{\gamma,i}$ to $R'_{\gamma,i,j}$. Recall that this denotes the γth internal list which P_j queries to P_i. We now give the more technical description of our sequence of hybrid experiments:

- Let hybrid H_0 correspond to the ideal world experiment where $\mathcal{S}$ simulates the execution of Π, given access to the ideal functionality $\mathcal{F}_{\mathsf{pull}}$. Note that by definition, the output distribution of H_0 is equivalent to the ideal world distribution.
- Define hybrid $H^b_{1,2}$ as a modified version of H_0 as follows. The experiment can observe exactly what requests $R_{\gamma,i,j}$ are being generated in $\mathcal{F}_{\mathsf{pull}}$ for each party P_j to each other party P_i and how they are responded to via writes to the list $S_{j,i}$. This information is ignored and the experiment behaves identically as H_0, except for messages that $\mathcal{S}$ sends on behalf of honest P_1 to honest P_2. In this case, the experiment instead replaces the ciphertext in these message as follows. If $b = 1$, then for the Request message sent from P_1 to P_2, the experiment creates, for all $\gamma \in [c]$, the Request lists as generated in $\mathcal{F}_{\mathsf{pull}}$ on behalf of P_1 instead of using the values generated by $\mathcal{S}$. More precisely, the experiment forms the Step Two Request message from P_1 to P_2 by computing, for all $\gamma \in [c]$, $Reqs[\gamma][2] = \mathsf{ENC}(pk_{1,1,2}, (R'_{\gamma,2,1}, \mathsf{Ord}'_{\gamma,2,1}))$. Here, $R'_{\gamma,2,1}$ are directly taken from the internal state of $\mathcal{F}_{\mathsf{pull}}$. Moreover,

$\mathsf{Ord}'_{\gamma,2,1}$ is computed as follows. Recall that, in order to decide which indices will be transferred to the output list S_1 of P_1 from P_2's input list K_2, $\mathcal{F}_{\mathsf{pull}}$ internally uses $\mathsf{OrderRel}_1$. $\mathsf{OrderRel}_1$ in turn internally runs $\mathsf{OrderIn}$ and $\mathsf{OrderOut}$. Thus, for each $\gamma \in [c]$ and for each $k \in [\lceil \lambda / \log(n) \rceil]$, the experiment sets $\mathsf{Ord}_{\gamma,2,1}[m] = \mathsf{OrderIn}_1(M_1[R_{\gamma,2,1}[k]; \rho^1_{\gamma,m})$. Here, $\rho^1_{\gamma,1}, ..., \rho^1_{\gamma,\lceil \lambda/\log(n) \rceil}$ are the random coins for $\mathsf{OrderIn}_1$ used in $\mathcal{F}_{\mathsf{pull}}$ (via $\mathsf{OrderRel}_1$) to determine the minimum index k^1_γ for which $\mathsf{OrderRel}(K_2[R'_{\gamma,2,1}[k^1_\gamma]], M_1[R'_{\gamma,2,1}[k^1_\gamma]]) = 1$. We stress that we do not alter the coins that the experiment uses to sample messages from P_2 to P_1 via $\mathcal{S}$.

- The next hybrid in the sequence is $H^1_{2,1}$, which is defined analogously as $H^1_{1,2}$, but with indices reversed.
- For $H^2_{1,2}$ the experiment additionally modifies how the RESPONSE messages from P_1 to P_2 are formed. Here, the experiment simply sets, for each $\gamma \in [c]$ $Resp_1[2][\gamma] = \mathsf{ENC}(pk_{2,1,2}, K_1[R'_{\gamma,2,1}[k^2_\gamma]])$, where k^2_γ is the minimum index determined by $\mathcal{F}_{\mathsf{pull}}$ within $R'_{\gamma,2,1}$ such that $\mathsf{OrderRel}_1(K_1[R'_{\gamma,2,1}[k^2_\gamma]], M_2 R'_{\gamma,2,1}[k^2_\gamma]]) = 1$. These messages are sent as the RESPONSE message from P_1 to P_2 instead of what $\mathcal{S}$ would have sent. Note that at this point, the messages transferred from P_1 to P_2 via the RESPONSE message now match exactly what P_1 transfers to P_2 in $\mathcal{F}_{\mathsf{pull}}$.
- The next hybrid in the sequence is $H^2_{2,1}$, which, again, is defined analogously as $H^1_{1,2}$, but with indices reversed.
- More generally, for $1 \leq i < j \leq n, b \in \{0,1\}$, we define hybrid $H^b_{i,j}$ as a further modified version of H_0. This is done by implementing analogous changes to the ones made in the hybrids $H^b_{1,2}$, except that they are made for parties P_i and P_j (on top of the changes that were made in lexicographically prior hybrids). As described above, our sequence of hybrids first switches all messages sent between P_1 and other parties, then moves to P_2, and so on.
- Notice that by construction, the output distribution of the final hybrid in this sequence, $H^2_{n-1,n}$, corresponds exactly to that of $\mathbf{Real}(1^\kappa, \mathbf{x}, z)$; every REQUEST message sent from an honest sender to an honest receiver contains the encryption under the receiver's public key of the actual request list sent from the honest sender to the receiver. Every RESPONSE message sent from an honest sender to an honest receiver contains the encryption under the receiver's public key of the actual messages and indices sent from the sender to the receiver. Moreover, due to our changes in the hybrid games, the outputs of all honest parties (which are still given by $\mathcal{F}_{\mathsf{pull}}$!) match exactly with the messages that are sent as part of REQUEST and RESPONSE messages in the experiment.

We show that the output distributions of the real and ideal world experiments are computationally indistinguishable, assuming a CPA-secure PKE scheme $\mathsf{PKE} = (\mathsf{KeyGen}, \mathsf{ENC}, \mathsf{DEC})$ and secure erasure. We argue how the output distributions of any two consecutive hybrids are computationally indistinguishable under these assumptions. We show the main idea of the reduction for the case of H_0 to $H^0_{1,2}$; the rest of the cases follow analogously.

We describe a reduction $\mathcal{C}$ from the CPA security of PKE that turns an efficient distinguisher $\mathcal{Z}$ against H_0 and $H^1_{1,2}$ into a distinguisher w.r.t. CPA security.

The reduction $\mathcal{C}$. The reduction $\mathcal{C}$ plays in either one of the CPA indistinguishability games $\mathsf{CPA}^{\mathcal{C}}_{\mathsf{PKE},0}$ or $\mathsf{CPA}^{\mathcal{C}}_{\mathsf{PKE}}$.Thus, it obtains initially a public key pk and uses this key as the key $pk_{1,2,1}$ to simulate either H_0 or $H^b_{1,2}$ as follows:

1. It honestly samples all public keys of honest parties in known fashion.
2. It carries out the simulation exactly as $\mathcal{S}$, but internally also simulates the behavior of $\mathcal{F}_{\mathsf{pull}}$. In addition, it modifies the simulation between P_1 and P_2 as described in the following. The first modification lies in Step One, where it replaces the key $pk_{1,2,1}$ the key pk obtain from the challenger of the CPA experiment.
3. When the honest P_1 inputs to $\mathcal{F}_{\mathsf{pull}}$, $\mathcal{C}$ simulates the behavior of $\mathcal{F}_{\mathsf{pull}}$ as specified by the interface.
   $\mathcal{C}$ generates REQUEST message from P_1 to P_2 as specified by $\mathcal{S}$, and stores these messages as m_0. $\mathcal{C}$ also creates a REQUEST message using the internal information from $\mathcal{F}_{\mathsf{pull}}$, exactly as specified in the description of $H^1_{1,2}$. It stores this message as m_1 Now, $\mathcal{C}$ proceeds to hand to the challenger in its own CPA experiment both m_0 and m_1. When it receives a challenge ciphertext c, it uses this as the ciphertext (i.e., the REQUEST message) sent from P_1 to P_2 in its simulation.
4. After the simulation completes, $\mathcal{C}$ outputs the resulting output vector of the honest parties as output by $\mathcal{F}_{\mathsf{pull}}$, as well as the view of the set of corrupted parties to the distinguisher $\mathcal{Z}$. It receives a guess b from $\mathcal{Z}$.
5. $\mathcal{C}$ returns b to the CPA challenger.

If $\mathcal{C}$ played in the experiment $\mathsf{CPA}_{\mathsf{PKE},0}$, then the challenge ciphertext c corresponds to a request message as sent in H_0 and $\mathcal{C}$ perfectly simulates the output distribution of H_0. Otherwise, it perfectly simulates $H^1_{1,2}$. In particular, due to the erasure of keys $\mathcal{C}$ can efficiently simulate the experiment without having to output the secret key corresponding to pk. Thus,

$$|\Pr[H^{1,\mathcal{Z}}_{1,2} = 1] - \Pr[H^{\mathcal{Z}}_0 = 1]| =$$
$$|\Pr\left[\mathsf{CPA}^{\mathcal{C}}_{\mathsf{PKE},1} = 1\right] - \Pr\left[\mathsf{CPA}^{\mathcal{C}}_{\mathsf{PKE},0} = 1\right]| \leq \mathsf{negl}(\kappa).$$

Similarly, we obtain

$$|\Pr[H^{1,\mathcal{Z}}_{2,1} = 1] - \Pr[H^{1,\mathcal{Z}}_{1,2} = 1]| =\leq \mathsf{negl}(\kappa).$$

Note that in $H^1_{2,1}$, the REQUEST messages sent between P_1 and P_2 are computed using the same permutations and the same randomness as how $\mathcal{F}_{\mathsf{pull}}$ determines (via $\mathsf{OrderIn}_1$ in $\mathsf{OrderRel}_1$) which elements should be transferred in between P_1 and P_2. However, the RESPONSE messages do not yet match the elements that are decided by $\mathcal{F}_{\mathsf{pull}}$ (via $\mathsf{OrderOut}_1$ in $\mathsf{OrderRel}_1$). But because the REQUEST messages are now in line with what $\mathcal{F}_{\mathsf{pull}}$ actually transfers between P_1 and P_2,

it is not hard to see that we can switch the RESPONSE messages between P_1 and P_2 in the same fashion as done above, i.e.:

$$|\Pr[H_{2,1}^{1,\mathcal{Z}} = 1] - \Pr[H_{1,2}^{2,\mathcal{Z}} = 1]| \leq \mathsf{negl}(\kappa).$$

and

$$|\Pr[H_{1,2}^{2,\mathcal{Z}} = 1] - \Pr[H_{2,1}^{2,\mathcal{Z}} = 1]| \leq \mathsf{negl}(\kappa).$$

Continuing this line of reasoning across all $2n^2$ hybrids, we obtain that for all $\mathcal{Z}$

$$|\Pr[H_0^{\mathcal{D}} = 1] - \Pr[H_{n-1,n}^{2,\mathcal{Z}} = 1]| \leq 2n^2\mathsf{negl}(\kappa) = \mathsf{negl}(\kappa).$$

D Analyzing Distribute Certificate

Negative Association

Our analysis requires concentration bounds on random variables that are not fully independent. We use the following generalization of independence.

Definition 14 (Negative Association). *A set of random variables $X_1, ..., X_s$ is negatively associated, if for any two disjoint index sets $T, U \subseteq [s]$ and two functions f, g that are either both increasing or both decreasing, it holds that*

$$\mathbb{E}[f(X_t \colon t \in T) \cdot g(X_u \colon u \in U] \leq \mathbb{E}[f(X_t \colon t \in T)] \cdot \mathbb{E}[g(X_u \colon u \in U].$$

We use the following general rules for inferring negative association.

Lemma 11 ([34], Observation 6). *Let X_i, $i \in I$, be independent zero–one random variables. Then X_i, $i \in I$, are negatively associated.*

Lemma 12 ([34], Lemma 7). *Let X_i, $i \in I$, be zero–one random variables such that $\sum_i X_i \leq 1$. Then X_i, $i \in I$, are negatively associated.*

Lemma 13 ([34], Lemma 9(i)). *The union of independent sets of negatively associated variables is negatively associated.*

Lemma 14 ([34], Lemma 9(ii)). *Increasing functions defined on disjoint subsets of a set of negatively associated random variables are negatively associated. The same holds true for decreasing functions.*

The key property of negatively associated random variables is that Chernoff-Hoeffding type bounds apply. We will make use of the following well-known variants of Chernoff's bound.

Theorem 5 (Theorem 5 in [34], using $2\delta/(2+\delta) \leq \log(1+\delta)$). *Let $X_1, ..., X_s$ be negatively associated $0-1$ variables. Denoting $X := \sum_{t=1}^{s} X_t$ and $\mu := \mathbb{E}[X]$, we have that*

(i) $\Pr[X \geq (1+\delta)\mu] \leq e^{-\delta^2\mu/(\delta+2)}$ *for all* $\delta \geq 0$ *and*
(ii) $\Pr[X \leq (1-\delta)\mu] \leq e^{-\delta^2\mu/2}$ *for all* $0 \leq \delta \leq 1$.

Additional Notation

Recall that in our graded consensus protocol, each party signs their input, multicasts the signed input to all parties, and we form certificates from $t+1$ valid signatures. Trivially, this prevents forming such a certificate for a value that was not input to an honest party.

The goal of the Distribute Certificate procedure is to enable each party i that is not corrupted and has a certificate for value v to ensure that each honest party obtains a certificate for value v or a certificate proving that not all honest parties had the same input. The latter "special" certificate has a very similar structure if we choose for any two certificates for different values to form such a certificate. This means the above goal is to be rephrased as follows. If party i is not corrupted and inputs a certificate for v, then each honest party j obtains

(i) a certificate for v or
(ii) certificates for two different values.

It would be straightforward to spread the certificates by gossiping, if it was not for the fact that malicious parties can introduce valid elements for different values. Naively gossiping these would result in too high complexity. Our way out is the observation that conflicting elements from the same party prove equivocation of this party. Hence, it is safe to consider such a pair as valid for *all* possible values, enabling to efficiently prove to others that the signing party supported a given value despite the competing elements. This is captured by the following definition.

Definition 15 (Support). *We say that party i* supports v on index k *if it has*
- *a single element (σ, v) or*
- *a double element.*

The element is said to support v as well.

With this notation, party i holding a certificate for value v is equivalent to i supporting v on at least $t+1$ values, cf. Definitions 1 and 2

In order to ensure that certificate holders can spread their certificates, they spend extra effort in Step 3 of the loop in Distribute Certificate.

Definition 16 (Advertisement). *We say that party i* advertises certificate C *if in Distribute Certificate it has set one of its variables C_b, $b \in \{0,1\}$, to C.*

Note that each party advertises only the first two certificates it obtains; as suggested above, this is sufficient to achieve the guarantees we need.

In our analysis, we will first argue that *almost* all honest parties gain support for a given value v, which we express as follows.

Definition 17 (Widely Known Support). *For a value v and an index set $K \subseteq [n]$, we say that K* is v-widely known *if all but λ honest parties support v on each $k \in K$.*

Our analysis groups the loop iterations of Distribute Certificate into phases of $O(\log n)$ iterations each.

Definition 18 (Iterations and Phases). *Throughout this section, an* iteration *refers to an iteration of the loop in Distribute Certificate. A* phase *consists of $O(\log(n))$ iterations, where the constants in the O-notation are sufficiently large to accommodate the constants in the proofs of this section.*

Since the adversary is adaptive, we need to keep track of which parties remain honest by a given iteration r. The security guarantees of $\mathcal{F}_{\mathsf{pull}}$ leak information to the adversary at specific times. We need to define which parties we consider honest for a given iteration such that parties that learned elements get the chance to gossip about them once before the adversary is informed about what they learned. Otherwise, the adversary could determine which parties obtained a given element and corrupt them *before* the information spreads, breaking the protocol.

Definition 19 (Honest Sets). *Fix a phase p. By H_r we denote that set of parties i that are honest until K_i is output to the adversary in the first execution of $\mathcal{F}_{\mathsf{pull}}$ of iteration r (i.e., during Step 2 of this iteration) of phase p. By $H_{r,v,k}$ we denote the subset of H_r supporting v on index k at the beginning of iteration r of phase p. As p will always be clear from context, we omit it from the notation.*

In our analysis, we need to attribute obtaining elements to parties providing them to define relevant random variables.

Definition 20 (Pulling). *We say that party i* pulls support for value v on index k from party j in iteration r *if there is a call to $\mathcal{F}_{\mathsf{pull}}$ in iteration r in which $S_{i,j}$ contains an entry supporting v on k.*

Definition 21 (With High Probability). *For a parameter λ, we say that an event occurs w.h.p. if it does so with probability $1 - \mathsf{negl}.(\lambda) = 1 - 2^{-\Omega(\lambda)}$.*

Throughout most of this section, we assume that $C\log(n) \leq \lambda \leq n/C$ for a sufficiently large constant C. The lower bound is required to enable the use of union bounds over polynomially in n many events that occur w.h.p. to infer that they all occur jointly w.h.p. The upper bound guarantees that concentration in n suffices to show that something occurs w.h.p. At the end of the section, we will discuss how to boost success probability by concurrent execution to remove the upper bound.

Helper Statements

We start by proving some crucial helper statements. First, we show that for each value v and each index k, each iteration ensures that support for v on k spreads as least like in a "standard" gossip procedure in which on each link, the rumor is propagated with probability $\Omega(c/n)$.

Lemma 15. *If $i \in H_r$ supports v on k in iteration r, then $j \in H_{r+1}$ not supporting v on k before iteration r pulls support for v on k from i in iteration r with probability at least $1 - (1 - 1/n)^{\Omega(c)}$.*

Proof. As R_j is a uniformly random permutation on n indices, with probability $1/n$ we have that $k = R_j[\gamma][i]$ in the first call to $\mathcal{F}_{\mathsf{pull}}$ in iteration r, independently for each $\gamma \in [c]$. As j does not support v on k, $\mathsf{OrderRel}_1$ indicates 1 when comparing the values of i and j on that index with probability at least $1 - 1/n$: if i has a double element or j has $\perp$ for index k, this holds deterministically; if both i and j store a single element for index k, they are for different values, so the equality test reveals this with probability at least $1 - 1/n$.

Thus, $S_{i,j}$ contains the element that i stores for index k, and after updating $A_i[k]$, i supports v on index k. Due to independence for different values of γ, j pulls support for v on k from i with probability at least $1-(1-(1-1/n)/n)^c = 1-(1-1/n)^c$.

Next, for a value v consider the set K of indices such that for each $k \in K$, all but $n/4$ honest parties (possibly different sets for different k) support v on k. We show that K becomes widely known within $O(\log(n))$ iterations w.h.p.

To this end, For each honest party i, the values v of each of its advertised certificates, and each index k for which v is known to all but $n/4$ honest parties at the beginning of some iteration r_0 of a phase p such that $C\log(n)$ iterations remain for a suitable constant C, denote by $I_{v,j,k,p}$ an indicator variable that is 1 if j is neither corrupted nor supports v on k by the end of the phase.

Lemma 16. *W.h.p.,* $\sum_{v,j,k,p} I_{v,j,k,p} \leq \lambda$.

Proof. Fix a party j. If j gets corrupted at any point before phase p is complete, $I_{v,j,k,p} = 0$, so assume in the following that j remains honest. For any v and k such that j does not support v on k at the beginning of iteration r_0, each iteration $r \geq r_0$ of the phase, and each $r' \in H_r$, denote by $X_{j,r',\gamma,v,k,r}$, where $\gamma \in [1,...,c]$, an indicator variable that is 1 if and only if j pulls support for v on k from r' in iteration r and $\mathsf{OrderRel}_1(K_{r'}[R_j[\gamma][r']][k]], M_{r'}[R_j[\gamma][r']][k]]) = 1$.

For fixed r, the variables $X_{j,r',\gamma,v,k,r}$ are negatively associated:

- For fixed r' and γ, $\sum_{v,k} X_{j,r',\gamma,v,k,r} \leq 1$, so these variables are negatively associated by Lemma 12.
- For different r' or different γ, the respective sets of variables are independent. Hence, for fixed r the variables $X_{j,r',\gamma,v,k,r}$ are negatively associated by Lemma 13.

If we could extend this property to all $X_{j,r',\gamma,v,k,r}$ (i.e., for varying r), we could apply Lemma 14 to infer that $I_{v,j,k,p} = 1$, being increasing functions of $X_{j,r',\gamma,v,k,r}$, are negatively associated.

Unfortunately, the precise sets of known signatures vary with r and affect the pulling probabilities. We now deal with this by a domination argument suppressing the dependency on the iteration r. We make three key observations:

- If r' supports v on k, but j does not, by Lemma 15, j pulls support for v on k from r' with probability at least $1-(1-1/n)^{\Omega(c)} \geq 1/n$, where we use that c is a sufficiently large constant.

– As v is known to all but $n/4$ parties, regardless of corruptions we have that $|H_{r,v,k}| \geq t + 1 - n/4 \geq n/4$.
– Once j supports v on k, increasing the probability for j to pull support for v on k does not influence $I_{v,j,k,p}$, which then is already determined to be 0.

Therefore, we can replace $X_{j,r',\gamma,v,k,r}$ by negatively associated $0-1$ variables $Y_{v,k,r,\alpha}$, where $\alpha \in [n/3]$ and $\Pr[Y_{v,k,r,\alpha} = 1] \geq 1/n$: in each iteration in which j has not yet learned signatures causing it to support v on k, there are at least $n/4$ honest parties doing so, and hence we can map each $Y_{v,k,r,\alpha}$ to a distinct $X_{j,r',\gamma,v,k,r}$ with $\Pr[X_{j,r',\gamma,v,k,r} = 1] \geq (1 - 1/n)/n$; on the other hand, once j supports v on k, adding the same variables $Y_{v,k,r,\alpha}$ as if it would not do so yet maintains negative association and does not affect $I_{v,j,k,p}$.

Now define $I'_{v,j,k,p}$ as the indicator variable that is 1 if and only if j is not corrupted and all $Y_{v,k,r,\alpha}$ for varying r and α are 0. By our reasoning above we have established that (i) $\sum_{v,j,k,p} I'_{v,j,k,p}$ dominates $\sum_{v,j,k,p} I_{v,j,k,p}$ from above and (ii) as decreasing functions of disjoint subsets of the variables $Y_{v,k,r,\alpha}$, the variables $I'_{v,j,k,p}$ are negatively associated by Lemma 14. The second property enables us to apply Theorem 5 to $\sum_{v,j,k,p} I'_{v,j,k,p}$. If we can show that $\mu := \mathbb{E}[\sum_{v,j,k,p} I'_{v,j,k,p}] \leq 1$, using (i) and choosing[8] $\delta := (\lambda - \mu)/\mu = \Omega(\lambda/\mu)$ in Theorem 5(i) yields that

$$\begin{aligned}
\Pr\left[\sum_{v,j,k,p} I_{v,j,k,p} > \lambda\right] &\leq \Pr\left[\sum_{v,j,k,p} I'_{v,j,k,p} > \lambda\right] \\
&= \Pr\left[\sum_{v,j,k,p} I'_{v,j,k,p} > (1+\delta)\mu\right] \\
&\leq e^{-\delta^2\mu/(2+\delta)} \\
&\leq e^{-\Omega(\lambda)},
\end{aligned}$$

which completes the proof.

To see that indeed $\mu \leq 1$, recall that whether j pulls support for v on k from r' in iteration r is independent for different r', and we eliminated dependencies between iterations when defining the variables $Y_{v,k,r,\alpha}$. Therefore,

$$\Pr[I'_{v,j,k,p} = 1] = \prod_{r,\alpha} 1 - \Pr[Y_{v,k,r,\alpha} = 0] = \prod_{r} \left(1 - \frac{1}{n}\right)^{n/4} = \prod_{r} 1 - \Omega(1).$$

Given that a phase has $C \log n$ iterations for sufficiently large C, it follows that $\Pr[I'_{v,j,k,p} = 1] \leq n^{-3}/2$. There are at most n different parties j, n different indices k, and $2n$ different values v (as each honest party advertises at most two values) to consider, i.e., in total we have no more than $2n^3$ variables $I'_{v,j,k,p}$. We conclude that $\mu \leq 1$ and thus the proof.

[8] Here, we assume w.l.o.g. that $\lambda \geq 2$.

As our final helper lemma, we show a stronger bound on the probability to pull support on advertised values.

Lemma 17. *If $i \in H_{r+1}$ advertises a certificate for v comprising support for v on index k, then $j \in H_{r+1}$ not supporting v on k before iteration r pulls support for v on k from i in iteration r with probability $\Omega(\min\{c/x, c\lambda/(n\log(n))\}$, where x is the number of indices in i's certificate for v which j does not support v.*

Proof. We consider the instance of $\mathcal{F}_{\mathsf{pull}}$ in iteration r for which i answers only from its certificate C_b, $b \in \{0,1\}$, for v. Let X denote the set of indices on which i's certificate supports v but j does not, i.e., $x = |X|$.

We claim that, independently for each $\gamma \in [c]$,

$$\Pr[\exists \ell \in X \text{ s.t. } \ell \in R_{\gamma,i}] \geq 1 - \left(1 - \frac{\lambda}{n\log(n)}\right)^x. \tag{1}$$

This holds true, because $R_{\gamma,i}$ is a prefix of length $\lambda/\log(n)$ of a uniformly random permutation of $[n]$, and for the ℓ-th element of X, not receiving one of the first $\lambda/\log(n)$ indices conditioned on the previous elements not doing so either has probability at most $1 - \lambda/(n\log(n))$.

Observe that the order of X in the permutation is uniformly random even when conditioning on the event $\mathcal{E}_1$ that that there is some $\ell \in X$ such that $\ell \in R_{\gamma,i}$. Therefore, the probability that, conditioning on $\mathcal{E}_1$, k is the index in X that appears first in $R_{\gamma,i}$ is $1/x$. Denote by $\mathcal{E}_2$ the event that this is the case.

Finally, note that the event $\mathcal{E}_3$ that $\mathsf{OrderRel}_2$ evaluates to 1 on index k is, due to i supporting v on k but j not, at least $1 - 1/n$. As this event occurs independently of $\mathcal{E}_1$ and $\mathcal{E}_2$ by definition, and there are c independent parallel attempts by j to pull support for v on k from i, it follows that j fails to do so with probability at most

$$(1 - \Pr[\mathcal{E}_1] \cdot \Pr[\mathcal{E}_2] \cdot \Pr[\mathcal{E}_3])^c \leq \left(1 - \left(1 - \left(1 - \frac{\lambda}{n\log(n)}\right)^x\right) \cdot \frac{1}{x} \cdot \left(1 - \frac{1}{n}\right)\right)^c.$$

To further bound this probability from above, we distinguish two cases.

1. $x \geq n\log(n)/\lambda$**.** Then $\lambda/(n\log(n)) \geq 1/x$ and hence

$$1 - \left(1 - \frac{\lambda}{n\log(n)}\right)^x \geq 1 - \left(1 - \frac{1}{x}\right)^x = \Theta(1),$$

where the second step uses that $(1 - 1/y)^y$ is increasing in y for $y \geq 1$ and converges to $1/e$. It follows that

$$(1 - \Pr[\mathcal{E}_1] \cdot \Pr[\mathcal{E}_2] \cdot \Pr[\mathcal{E}_3])^c \leq (1 - \Theta(1/x))^c,$$

so using that c is constant and $x \gg 1$, the probability of the complementary event that j pulls support for v on k from i is $\Omega(c/x)$.

2. $x < n\log(n)/\lambda$**.** Using again that $(1-1/y)^y$ is increasing and converges to $1/e$, we infer that $\left(1-\frac{\lambda}{n\log(n)}\right)^{x'} \geq 1/2$ for all $0 < x' \leq x$. By induction on x', it follows that

$$\left(1-\frac{\lambda}{n\log(n)}\right)^{x} \leq 1-\frac{x\lambda}{2n\log(n)}.$$

Assuming w.l.o.g. that $n \geq 2$, this leads to

$$(1-\Pr[\mathcal{E}_1]\cdot\Pr[\mathcal{E}_2]\cdot\Pr[\mathcal{E}_3])^c \leq \left(1-\frac{\lambda}{4n\log(n)}\right)^{c} \leq 1-\Omega\left(\frac{c\lambda}{n\log(n)}\right),$$

where the last step again uses the above convergence and that $c \in O(1)$, $\lambda \in o(n\log(n))$. □

Spreading Certificates

We are now ready to move to the heart of our analysis of Distribute Certificate. We seek to show that if an honest party holds a certificate for value v and does not get corrupted, the subset of $t+1$ indices for which the certificate contains an element supporting v becomes v-widely known.

To this end, we argue over multiple phases. Denote by x_p an upper bound on the number of indices of the considered certificate that are not yet widely known by the beginning of phase p; trivially $x_1 \leq t+1$. We will show that for a large fraction of these indices, in phase p (i) the advertising party will distribute the respective elements supporting v to many parties, (ii) this will start an exponential growth of the number of parties newly obtaining support for v on such an index until all but $n/4$ honest parties support v on this index, and (iii) by applying Lemma 16, all of these indices become widely known.

To show that x_p w.h.p. decreases quickly with p as suggested above, fix a phase p and iteration r of phase p. Define

$$S_k := \{j \in H_{r+1} \setminus H_{r,v,k} \,|\, \text{at the beginning of iteration } r, \\ j \text{ supports } v \text{ on all but } x_p \text{ indices of the certificate}\};$$

note that $|S_k| \geq |H_{r+1} \setminus H_{r,v,k}| - \lambda$ w.h.p.

For (i) and (ii), we apply the same pattern of reasoning, but once use Lemma 17 and Lemma 15 to bound individual pulling probabilities. Accordingly, let q denote a lower bound for any party $j \in S_k$ to pull support for v on an index k of the certificate it does not support yet in one of the instances of $\mathcal{F}_{\mathsf{pull}}$ called in iteration r. For any $g \in [|S_k|/20]$ let $\mathcal{P}_{g,k}$ be a (fixed) maximal collection of disjoint subsets of S_k, each of size g. For each $\mathcal{A} \in \mathcal{P}_{g,k}$, define $X_{\mathcal{A},k}$ to be the indicator variable that evaluates to 1 if and only if fewer than $9qg/10$ parties in $\mathcal{A}$ pull support for v on k in iteration r.

The reason for this "grouping" of the parties is to use concentration over the (independent) pulling operations of the individual parties. However, regardless

of how small the probability to fail becomes, Theorem 5 does not guarantee that $o(\lambda)$ indicator variables become 1. Choosing g smaller than n, we can account for this by exploiting that $\Theta(n/g)$ variables need to become 1 for a single index k in order for much fewer than the expected number of parties pulling support for v on k. This balance is reflected by the two terms in Lemma 19 on the indices for which few parties pull support for v. First, however, we need to establish that we can apply Theorem 5 to the variables we specified.

Lemma 18. *The collection of all variables $\{X_{\mathcal{A},k}\}_{\mathcal{A},k}$ is negatively associated.*

Proof. Let $Y_{i,j,k,\gamma}$ indicate whether $j \in H_{r+1} \setminus H_{r,v,k}$ pulls support for v on k from $i \in H_{r,v,k}$ due to list $R_i[\gamma][j]$ in the considered call to $\mathcal{F}_{\mathsf{pull}}$ in iteration r. Observe that the variables $X_{\mathcal{A},k}$ are decreasing functions of disjoint subsets of the collection of variables $\{Y_{i,j,k,\gamma}\}_{i,j,k,\gamma}$, so by Lemma 14, it is sufficient to show that the latter collection is negatively associated.

To see this, note that for each i, j, and γ, it holds that $\sum_k Y_{i,j,k,\gamma} \leq 1$, so these variables are negatively associated by Lemma 12. Because the randomness used for each combination of i, j, and γ is independent, the groups of variables $\{\{Y_{i,j,k,\gamma}\}_k\}_{i,j,\gamma}$ are mutually independent, implying by Lemma 13 that the collection $\{Y_{i,j,k,\gamma}\}_{i,j,k,\gamma}$ is indeed negatively associated.

Lemma 19. *W.h.p., there exist at most $60e^{-qg/8}x_p + 30g\lambda/n$ indices k such that $|H_{r+1,v,k} \setminus H_{r,v,k}| < 3q(|S_k| - g)/4$ and $|H_{r+1} \setminus H_{r,v,k}| \geq n/4$.*

Proof. Denote by K the set of indices appearing in the certificate for v such that $|H_{r+1} \setminus H_{r,v,k}| \geq n/4$. For an index $k \in K$, $\mathcal{A} \in \mathcal{P}_{g,k}$, and party $j \in \mathcal{A}$, let $X_{\mathcal{A},k,j}$ denote the indicator variable for the event that j pulls support for v on k in the considered call to $\mathcal{F}_{\mathsf{pull}}$ during iteration r. We note that these variables are all independent and become 1 with probability q. For each $\mathcal{A}$ and each k, we have that

$$\Pr[X_{\mathcal{A},k} = 1] = \Pr\left[\sum_{j \in \mathcal{A}} X_{\mathcal{A},k,j} < 9qg/10\right].$$

Since $\mathbb{E}[\sum_{j \in \mathcal{A}} X_{\mathcal{A},k,j}] \geq qg$, Theorem 5(ii) with $\delta = 1/10$ yields that

$$\begin{aligned}\Pr\left[X_{\mathcal{A},k} = 1\right] &= \Pr\left[\sum_{j \in \mathcal{A}} X_{\mathcal{A},k,j} < 9qg/10\right] \\ &\leq \Pr\left[\sum_{j \in \mathcal{A}} X_{\mathcal{A},k,j} \leq \frac{9}{10} \cdot \mathbb{E}\left[\sum_{j \in \mathcal{A}} X_{\mathcal{A},k,j}\right]\right] \leq e^{-qg/200}.\end{aligned}$$

Let $X = \sum_{\mathcal{A},k} X_{\mathcal{A},k}$, where k ranges over all indices from K. Note that $|K| \leq x_p$, since any widely known index is known to all but $\lambda \ll n/4$ honest parties. It follows by linearity of expectation that $\mu := \mathbb{E}[X] \leq x_p \cdot e^{-qg/200} \cdot n/g$.

As we have shown in Lemma 18, the collection of all variables $X_{\mathcal{A},k}$ is negatively associated. Hence, we can apply Theorem 5(i) with $\delta = 1$ to obtain

$$\Pr[X \geq 2\mu + \lambda] \leq \Pr[X \geq 2\mu] \leq e^{-\mu/3}.$$

If $\mu \geq \lambda$, this bound holds with high probability in λ. Otherwise, we choose $\delta = \lambda/\mu$, resulting in

$$\Pr[X \geq 2\mu + \lambda] \leq \Pr[X \geq \mu + \lambda] \leq e^{-\lambda^2/(2\mu+\lambda)} \leq e^{-\lambda/3}.$$

Either way, $X \leq 2\mu + \lambda$ w.h.p.

Now observe that for each considered index k, the event that $|H_{r+1,v,k} \setminus H_{r,v,k}| < 3q(|S_k| - g)/4$ requires that sufficiently many indicators $X_{\mathcal{A},k}$ equal 1. We have at least $(|S_k| - g)/g$ indicators, and each indicator that is 0 guarantees $9qg/10$ distinct parties in $|H_{r+1,v,k} \setminus H_{r,v,k}|$, entailing that the event results in at least

$$\frac{|S_k| - g}{g} - \frac{3q(|S_k| - g)/4}{9qg/10} = \frac{|S_k| - g}{6g}$$

such indicators to be 1. As $k \in K$, $|H_{r+1,v,k} \setminus H_{r,v,k}| > n/4$ and hence

$$|S_k| - g \geq \frac{19|S_k|}{20} \geq \frac{19(|H_{r+1} \setminus H_{r+1,v,k}| - \lambda)}{20} > \frac{19n/4 - \lambda}{20} > \frac{9n}{40}.$$

Thus, w.h.p. at most

$$\frac{80g(2\mu + \lambda)}{3n} < 60e^{-qg/8}x_p + \frac{30g\lambda}{n}$$

indices can exceed the stated bound, completing the proof.

Next we use this lemma to show that (i) and (ii) occur for sufficiently many indices, provided that the pulling probability we get from Lemma 17 is $\Omega(c/x_p)$; this is the region in which exploiting concentration over multiple indices is crucial.

Lemma 20. *Suppose that $x_p = \Omega(n \log(n)/\lambda)$. Then, w.h.p.,*

$$x_{p+1} \leq \begin{cases} e^{-2n/x_p}x_p + O(\lambda) & \text{if } x_p \geq C\lambda \\ O(x_p \cdot \lambda/n \cdot \log(n/\lambda)) & \text{else,} \end{cases}$$

where C is a sufficiently large constant.

Proof. We seek to show that w.h.p. a large fraction of the indices from the considered certificate that are not yet v-widely known will be learned by all but $n/4$ honest parties within $O(\log n)$ iterations. To this end, we claim that

1. in iteration 1 of the phase, w.h.p. all but

$$\begin{cases} e^{-\Omega(cn/x_p)}x_p + O(\lambda) & \text{if } x_p \geq C\lambda \\ O(x_p \cdot \lambda/n \cdot \log(n/\lambda)) & \text{else} \end{cases} \tag{2}$$

   of them become newly supported for v by n/x_p fresh honest parties or all but $n/4$ honest parties, and
2. in iterations $r \geq 2$, w.h.p. all but

$$\begin{cases} e^{-\Omega(1.5^{r-2}cn/x_p)}x_p + O(1.5^{2-r}\lambda) & \text{if } x_p \geq C\lambda \\ O(1.5^{2-r}x_p \cdot \lambda/n \cdot \log(n/\lambda)) & \text{else} \end{cases}$$

   of the ones that did not drop out become newly supported for v by n/x_p fresh honest parties or all but $n/4$ honest parties.

Noting that $2^{\lceil \log(n) \rceil} n/x_p > n$ and applying a union bound, w.h.p. the above bounds always hold, and within $O(\log(n))$ iterations, all remaining indices are supported for v by all but $n/4$ honest parties. Applying Lemma 16 and another union bound then shows that these indices become v-widely known within $O(\log(n))$ additional iterations w.h.p.

Summation over the above bounds yields a geometric series, showing that the total number of indices which do not become known to all but $n/4$ honest parties satisfies the asymptotic bound stated in Equation (2); using that c is a sufficiently large constant, this translates to the statement of the lemma. To complete the proof, it hence remains to prove the above two claims.

To see the first claim, recall that there is an honest party holding the certificate that does not get corrupted. Hence, in iteration 1 of the phase, by Lemma 17 any party $j \in H_2$ pulls support for v on a given index k for which it does not support v yet from the certificate holder with probability $q = \Omega(\min\{c/x_p, c\lambda/(n\log(n))\})$ (independently of any other party). By assumption, $q = \Omega(c/x_p)$. If $x_p \geq C\lambda$, we choose $g = n/100$ in Lemma 19,[9] yielding that all but

$$e^{-\Omega(cg/x_p)}x_p + O(g\lambda/n) = e^{-\Omega(cn/x_p)}x_p + O(\lambda)$$

indices satisfy that $|H_{r+1,v,k} \setminus H_{r,v,k}| \geq 3q(|S_k| - g)/4 = \Omega(c|S_k|/x_p)$. Thus, for each such k either $|H_{r+1} \setminus H_{r,v,k}| < n/4$, which implies the claim because $H_{r+1,v,k} \cap H_{r+1} \subseteq H_{r,v,k} \cap H - r + 1$, or $\Omega(c|S_k|/x_p) = \Omega(c|H_{r+1} \setminus H_{r,v,k}|/x_p) = \Omega(cn/x_p)$ new parties pull support for v on k, i.e., at least n/x_p, since c is a sufficiently large constant.

Otherwise, in the lemma we choose $g = x_p \log(n/\lambda) < C\lambda \log(n/\lambda) \leq n/100$, since $\lambda \ll n$. This results in the bound of

$$\begin{aligned} e^{-\Omega(cg/x_p)}x_p + O(g\lambda/n) &= e^{-\Omega(c\log(n/\lambda))}x_p + O(x_p\lambda/n \cdot \log(n/\lambda)) \\ &= O(x_p\lambda/n \cdot \log(n/\lambda)) \end{aligned}$$

[9] Note that it is required that $g \leq |S_k|/20$ only for indices in K in the proof of Lemma 19, for which $|S_k| \geq n/4 - \lambda > n/5$, so this choice of g is valid.

for the number of such indices.

We now turn to the second claim. Here, we apply Lemma 15, resulting in pulling probability $\Omega(c/n)$ from each honest party supporting an index the pulling party does not support.[10] Since we only consider indices which have gained sufficient new support in the previous iteration, for any index that neither dropped out nor became known to all but $n/4$ honest parties, this implies that $q = 1 - (1 - \Omega(c/n))^{2^{r-2}n/x_p} = 1 - e^{-\Omega(2^{r-2}c/x_p)} = \min\{19/20, \Omega(2^{r-2}c/x_p))$ is a valid lower bound on the pulling probability.

We choose to apply the lemma with group size $g_r := 1.5^{-(r-2)}g$, resulting in the bound on the number of indices that drop out being bounded as claimed. It remains to argue that all other indices, i.e., those for which

$$\begin{aligned}
|H_{r+1,v,k} \setminus H_{r,v,k}| &\geq \frac{3q(|S_k| - g)}{4} \\
&\geq \frac{3q(|H_{r+1} \setminus H_{r,v,k}| - \lambda - g)}{4} \\
&\geq \min\left\{\frac{2}{3}, \Omega\left(2^{r-2} \cdot \frac{c}{x_p}\right)\right\} \cdot |H_{r+1} \setminus H_{r,v,k}| \\
&\geq \min\left\{\frac{2}{3}, 2^{r-1} \cdot \frac{4}{x_p}\right\} \cdot |H_{r+1} \setminus H_{r,v,k}|,
\end{aligned}$$

where the lower bound on $|S_k|$ holds w.h.p. and the final step uses that c is a sufficiently large constant, meet the criteria of the claim.

We already observed that if $|H_{r+1} \setminus H_{r,v,k}| \leq n/4$, the claim holds. Otherwise, if the minimum equals $2^{r-1} \cdot 4/x_p$, the above bound is at least $2^{r-1}n/x_p$, as desired.

Finally, if the minimum equals $2/3$, we get that

$$\begin{aligned}
|H_{r+1} \setminus H_{r+1,v,k}| &= |H_{r+1} \setminus H_{r,v,k}| - |H_{r+1,v,k} \setminus H_{r,v,k}| \\
&\leq \frac{|H_{r+1} \setminus H_{r,v,k}|}{3}.
\end{aligned}$$

If $|H_{r+1} \setminus H_{r+1,v,k}| \leq n/4$, the claim holds, so assume otherwise. Then it follows that $|H_{r+1} \setminus H_{r,v,k}| \geq 3n/4$, implying that $|H_{r,v,k}| \leq n/4$. By the bound from iteration $r-1$, we conclude that

$$\frac{2^{r-2}n}{x_p} \leq |H_{r,v,k} \setminus H_{r-1,v,k}| \leq |H_{r,v,k}| \leq \frac{n}{4}.$$

Putting things together, we obtain that

$$|H_{r+1,v,k} \setminus H_{r,v,k}| \geq \frac{2|H_{r+1} \setminus H_{r,v,k}|}{3} \geq \frac{n}{2} \geq \frac{2^{r-1}n}{x_p},$$

as desired.

[10] Here it is crucial that sending operations are atomic. The adversary must choose which parties are in H_{r+1} *before* learning which of them successfully pull support for v on k. Thus, the pulling probability does not depend on membership in H_{r+1}.

Corollary 6. *For $p = O(\log(n)/\log(n/\lambda))$, w.h.p. $x_p = O(n\log(n)/\lambda)$.*

Proof. So long as $x_p \neq O(n\log(n)/\lambda)$, we can repeatedly apply Lemma 20. While also $x_p \neq O(\lambda)$, the first case applies, i.e.,

$$x_{p+1} \leq e^{-2n/x_p} x_p + O(\lambda).$$

While the first summand dominates, this entails that $x_{p+1} < 2^{-n/x_p} x_p$. This can be restated as $n/x_{p+1} > 2^{n/x_p} \cdot n/x_p > 2^{n/x_p}$, i.e., n/x_p grows at least as fast as a tower in p. Thus, $x_p = O(\lambda)$ for $p \geq 1 + \log^*(n/\lambda)$.

Now assume that p is large enough such that $x_p = O(\lambda)$. Then $x_{p+1} = x_p \cdot O(\lambda/n \cdot \log(n/\lambda) =: x_p \cdot b$. As $\lambda \ll n$, $b < 1$, and it follows that $x_{p'} = O(n\log(n)/\lambda)$ for $p' = p + \log_b(n\log(n)) = p + O(\log(n)/\log(n/\lambda))$. Since $\log^*(\lambda)\log(n/\lambda) = O(\log(n))$, the claim follows.

Corollary 6 shows that within $O(\log(n)/\log(n/\lambda))$ phases, x_p will become $O(n\log(n)/\lambda)$. We now prove that the remaining indices become v-widely known within one more phase. Here, we run into the limitation that the pulling probability guaranteed by Lemma 17 is capped at $\Theta(\lambda/(n\log(n))$. However, this entails that even for a single index, over the course of $O(\log(n))$ iterations, w.h.p. $\Theta(\lambda)$ parties pull support for a given index from the certificate holder. This is good enough to prove a w.h.p. bound for each individual index.

Lemma 21. *Fix an index k for which at least one party that remains honest and advertises a certificate for v with support on k throughout phase p. If $x_p = O(n\log(n)/\lambda)$, w.h.p. k is v-widely known by the end of the phase.*

Proof. Denote as q_r an upper bound on the probability for the event that after r consecutive iterations of phase p, v is not supported on k by more than $n/4$ honest parties. Here, q_r holds under worst-case conditions under the constraint that the preconditions of the lemma are met. That is, we assume that the adversary may pick any j consecutive iterations during phase p and has full control over the signatures the parties hold initially, so long as the assignment does not modify the certificate or result in x_p being too large.

Let $q := 2^{-\Omega(\lambda/\log n)}$. We claim that, for all $r > 1$ satisfying that $q_{r-1} \geq 2^{-\lambda}$, it holds that $q_r \leq \max\{(2q)^{r-\lceil\log n\rceil}, 2^{-\lambda}\}$. We prove this statement by strong induction on j satisfying $p_{r-1} \geq 2^{-\lambda}$. For the base case, we note that the statement is trivially true when $r \leq \lceil\log n\rceil$, since $2q < 1$ and hence $(2q)^{r-\lceil\log n\rceil} \geq 1$.

We proceed with the step case. Hence, assume that $r > \lceil\log n\rceil$ and $p_{r-r'} \geq 2^{-\lambda}$ for all $1 \leq r' < r$. By the induction hypothesis, for all $1 \leq r' < r$, we have $q_{r-r'} \leq \max\{(2q)^{r-r'-\lceil\log(n)\rceil}, 2^{-\lambda}\}$, which can be simplified to $q_{r-r'} \leq (2q)^{r-r'-\lceil\log(n)\rceil}$.

For iteration r of phase p, denote as $\mathcal{E}_r$ the event that $|H_r \setminus H_{r,v,k}| \leq n/4$ or

$$\begin{cases} |H_{1,v,k}| \geq 1 & \text{if } r = 1 \\ |H_{2,v,k} \setminus H_{1,v,k}| \geq \lambda/\log(n) & \text{if } r = 2 \\ |H_{r,v,k} \setminus H_{r-1,v,k}| \geq 2^{r-2}\lambda/\log(n) & \text{else.} \end{cases}$$

Note that $\mathcal{E}_1$ is guaranteed by assumption and that $\mathcal{E}_{r,\lceil\log(n)\rceil}$ implies that all but $n/4$ honest parties support v on k, since there are only n honest parties.

Observe that by the definition of q_r, for any $r > r'$, $q_{r-r'}$ is an upper bound on the conditional probability that more than $n/4$ honest parties do not support v on k by iteration r for any event (that can occur with non-zero probability given the preconditions of the lemma).

We partition the probability space into the mutually exclusive events $\bar{\mathcal{E}}_r \wedge \bigwedge_{1\leq r'<r} \mathcal{E}_{r'}$ for $0 \leq r \leq \lceil\log(n)\rceil$ and $\bigwedge_{1\leq r'\leq\lceil\log(n)\rceil} \mathcal{E}_{r'}$. As $\Pr[\bar{\mathcal{E}}_0] = 0$ and $\mathcal{E}_{\lceil\log(n)\rceil}$ guarantees that all but $n/4$ honest parties support v on k by iteration $\lceil\log(n)\rceil$, the law of total probability yields that

$$\begin{aligned} q_r &\leq \sum_{1\leq r'\leq\lceil\log(n)\rceil} \Pr\left[\bar{\mathcal{E}}_{r'} \mid \bigwedge_{0\leq r''<r'} \mathcal{E}_{r''}\right] \cdot p_{r-r'} \\ &\leq \sum_{1\leq r'\leq\lceil\log(n)\rceil} \Pr\left[\bar{\mathcal{E}}_{r'} \mid \mathcal{E}_{r'-1}\right] \cdot p_{r-r'}. \end{aligned}$$

Claim. $\Pr\left[\bar{\mathcal{E}}_2 \mid \mathcal{E}_1\right] \leq q$.

Proof. By definition, the total number of honest parties that do not support some v-widely known index by the beginning of phase p is at most λ. For $\bar{\mathcal{E}}_2$ to possibly occur, at least $n/4$ honest parties must not yet support v on k. By the preconditions of the lemma, we can apply Lemma 17 to show that each such party pulls support for v on k from the certificate holder with probability $\Omega(c\lambda/(n\log(n)))$ in the first iteration of phase p, which holds independently from other parties; using that c is a sufficiently large constant, this probability is at least $10\lambda/(n(\log(n))$. Thus, in expectation at least $(n/4-\lambda)\cdot 10\lambda/\log(n) > 2\lambda/\log(n)$ such parties pull support for v on k in iteration 1. By Theorem 5(ii) with $\delta = 1/2$, thus with probability at most $e^{-\lambda/(4\log(n))} \leq q$, fewer than $\lambda/\log(n)$ such parties do so.

Claim. For $r > 1$, $\Pr\left[\bar{\mathcal{E}}_r \mid \mathcal{E}_{r-1}\right] \leq \min\{q^{2^r}, 2^{-\Omega(n)}\}$.

Proof. By Lemma 15, each party in $H_r \setminus H_{r-1,v,k}$ pulls support for v on k from each party in $H_{r-1,v,k}$ with (independent) probability[11] at least $1-(1-1/n)^{\Omega(c)}$. As $\mathcal{E}_{r-1}$ occurred, $|H_{r-1,v,k}| \geq 2^{r-2}\lambda/\log(n)$, implying that parties in $H_r \setminus H_{r-1,v,k}$ pull support for v on k with independent probability at least $1-(1-1/n)^{\Omega(c2^{r-2}\lambda/\log(n))} = 1-e^{-\Omega(c2^r\lambda/(n\log(n)))} = \min\{9/10, 8\cdot 2^r\lambda/(n\log(n))\}$, where we used that c is a sufficiently large constant.

We distinguish between the two cases in the minimum. If it equals $9/10$,

$$\mathbb{E}[|H_r \setminus H_{r,v,k}|] \leq \frac{1}{10}\cdot|H_r \setminus H_{r-1,v,k}| \leq \frac{n}{10}.$$

Applying Theorem 5(i) then shows that $|H_r \setminus H_{r,v,k}| \leq n/5$ with probability $e^{-\Omega(n)}$.

[11] Again, here it is essential that sending is atomic, so the adversary must choose H_r without knowledge on which parties pull support for v on k.

Otherwise, we use that $|H_r \setminus H_{r-1,v,k}| \geq n/4$ for $\bar{\mathcal{E}}_r$ to possibly occur, so

$$\mathbb{E}[|H_{r,v,k} \setminus |H_{r-1,v,k}|] \geq 8 \cdot \frac{2^r \lambda}{(n \log(n))} \cdot \frac{n}{4} = 2 \cdot \frac{2^r \lambda}{\log(n)}.$$

By Theorem 5(ii) with $\delta = 1/2$, thus $\bar{\mathcal{E}}_r$ occurs with probability at most $e^{-2^{r-3}\lambda/\log(n)} \leq q^{2^r}$.

Using these claim, we can now proceed as

$$\begin{aligned}
q_r &\leq \sum_{1 \leq r' \leq \lceil \log(n) \rceil} \Pr\left[\bar{\mathcal{E}}_{r'} \mid \mathcal{E}_{r'-1}\right] \cdot p_{r-r'} \\
&\leq \sum_{1 \leq r' \leq \lceil \log(n) \rceil} 2^{-\Omega(\min\{n, 2^{r'}\lambda/\log n\})} \cdot p_{r-r'} \\
&\leq \sum_{1 \leq r' \leq \lceil \log(n) \rceil} q^{2^{r'}} \cdot p_{r-r'} + \sum_{1 \leq r' \leq \lceil \log(n) \rceil} 2^{-\Omega(n)} \cdot p_{r-r'} \\
&\leq \sum_{1 \leq r' \leq \lceil \log(n) \rceil} q^{2^{r'}} \cdot (2q)^{r-r'-\lceil \log n \rceil} + 2^{-\Omega(n)} \\
&= (2q)^{r-\lceil \log n \rceil} \cdot \sum_{1 \leq r' \leq \lceil \log(n) \rceil} 2^{-r'} \cdot q^{2^{r'}-r'} + 2^{-\Omega(n)} \\
&\leq \max\{(2q)^{r-\lceil \log n \rceil}, 2^{-\Omega(n)}\} \leq \max\{(2q)^{j-\lceil \log n \rceil}, 2^{-\lambda}\},
\end{aligned}$$

completing the induction step.

Finally, by setting $r = 2\lceil \log(n) \rceil$, we see that $q_r = 2^{-\Omega(\lambda)}$, and hence all but $n/4$ honest parties support v on k by iteration r w.h.p. Applying Lemma 16 and another union bound, w.h.p. k is v-widely known by the end of the phase.

Corollary 7. *For $p = O(\log(n)/\log(n/\lambda))$, we have that $x_p = 0$ w.h.p. In particular, if party i is honest until the end p and advertised a certificate for v from the beginning of the call to Distribute Certificate, then all but λ honest parties have a certificate for v by the end of phase p.*

Proof. The first statement readily follows from Corollary 6 and Lemma 21 by a union bound. For the second, recall the definition of x_p: for fixed i advertising a certificate for v, x_p bounds the number of indices the certificate contains supports v on that are not v-widely known. In fact, Lemma 16 shows a stronger claim: w.h.p. in total λ variables indicating support for v on such an index may be 0, implying the second statement.

It remains to prove that the remaining up to λ parties obtain certificates as well. To this end, we distinguish between two cases. Either, many parties advertise the certificate they obtained for v, or they must all have at least two other certificates that prevent them from doing so.

Lemma 22. *Suppose all but λ honest parties hold a certificate for value v by the end of phase p. Then (i) $n/5$ honest parties advertising v will never be corrupted or (ii) $n/5$ honest parties advertising two values each will never be corrupted.*

Proof. Each honest party that holds a certificate for v during phase p either advertises it or advertises two values. Since $n/2 - \lambda > n/5$ by assumption, the claim follows.

We first consider Case (i) of Lemma 22. To get enough concentration, we again argue over all missing indices concurrently. This is greatly simplified by the fact that once an index is widely known, this cannot be changed by corruptions. So, regardless of which parties get corrupted, in each iteration each index has a good chance of being pulled.

Lemma 23. *If $n/5$ honest parties advertise v throughout a phase and each honest party supports v on at least $t + 1 - \lambda$ indices, then w.h.p. each honest party has a certificate for v by the end of the phase.*

Proof. Fix a party i that neither has a certificate for v nor is corrupted at the beginning of the phase. It is sufficient to show that i will acquire a certificate for v by the end of the phase or get corrupted w.h.p. A union bound over all such parties i then proves the claim. If at any point the adversary decides to corrupt i, the statement trivially holds for i. Thus, assume w.l.o.g. that the adversary does not corrupt i in the following.

We pair up iterations $r - 1$ and r for each even iteration r of the phase. We consider the r-th pair *good,* if (i) i pulls support for v on at least half of the indices it is missing for a certificate for v; otherwise, it is *bad.* Observe that within $O(\log n)$ good iteration pairs, i obtains a certificate for v.

Now consider a bad iteration pair. Note that in the second iteration of the pair, i will call $\mathcal{F}_{\mathsf{pull}}$ in Step 5 with input A'_i which contains all the elements i had obtained by the beginning of iteration $r - 1$. Moreover, by assumption there are at least $n/5$ honest parties that advertise v since there beginning of the phase. We iterate over these parties in the order in which the execution determines their responses to i in the call to $\mathcal{F}_{\mathsf{pull}}$ to which they input their certificate for v. If the iteration pair is bad, all of these responses satisfy that for at least half of the indices on which the responding party's certificate supports v, but A'_i does not, i has not pulled an element supporting the value in the previously evaluated responses or iteration $r - 1$. Note that this holds independently of which parties the adversary corrupts, since $\mathcal{F}_{\mathsf{pull}}$ does not leak whether i pulls a new element supporting the value before the call to $\mathcal{F}_{\mathsf{pull}}$ completes.

We claim that during a bad iteration pair, due to the above process i obtains at least $n/5$ times with a probability independently bounded from below by $\lambda/(n \log(n))$ one of the initially at most λ supporting elements it is missing to complete the certificates $v_1, ..., v_\ell$. To see this, note that for each $\gamma \in [c]$,

1. in the relevant call to $\mathcal{F}_{\mathsf{pull}}$, for any party i' advertising a certificate containing *some* index on which A'_i does not support v, $R_i[\gamma][i']$ contains such an index with probability at least $\lambda/(n \log(n))$,
2. if for at least half of such indices, i did not yet obtain a respective element, the probability for a new one to come earlier in $R_i[\gamma][i']$ than any already obtained one is at least $1/2$,

3. we ensured that the equality test succeeds with probability at least $1 - 1/n$, and
4. events (i)–(iii) are independent and in conjunction imply that i pulls one of the λ missing elements with probability at least $\lambda/(2n\log(n)) - 1/n$.

Since this happens independently for each γ and $\lambda \gg \log(n)$, we can infer a lower bound of $\lambda/(n\log(n))$ on the probability to pull support for the respective value on some new index.

To complete the proof, we now follow the above process for each iteration pair, determining whether it is good or bad and keeping tally of the number of relevant elements obtained during bad iterations. By the above discussion, in expectation each bad iteration contributes at least $n/5 \cdot \lambda/(n\log(n)) = \lambda/(5\log n)$ such elements. As the probability lower bound of $\lambda/(n\log(n))$ holds independently for each considered event, by Theorem 5(ii) with $\delta = 1/2$ within $10\log n$ bad iteration pairs, i learns λ such elements w.h.p. However, this entails that i obtains a certificate for v, as it was missing support on at most λ indices.

On the other hand, recall that there can be only $O(\log n)$ good iteration pairs until i obtains a certificate for v. As each iteration pair is either good or bad, this concludes the proof.

Case (ii) of Lemma 22 is treated in a very similar way, but we need to account for the fact that there are many different certificates. We accommodate this by modifying the definition of good iteration pairs by weighing progress in accordance with the number of parties advertising for the respective value. For bad iteration pairs, there is little change, since progress on the up to λ indices for which support might still be missing is shared across all relevant values.

Lemma 24. *If $n/5$ honest parties advertise two values each throughout a phase and each honest party is for all of these values together not supporting at most λ indices, then w.h.p. each honest party has certificates for two different values by the end of the phase.*

Proof. Fix a party i that neither has two certificates nor is corrupted at the beginning of the phase. It is sufficient to show that i will acquire two certificates by the end of the phase or get corrupted w.h.p. A union bound over all such parties i then proves the claim. If at any point the adversary decides to corrupt i, the statement trivially holds for i. Thus, assume w.l.o.g. that the adversary does not corrupt i in the following.

Denote by $v_1, \ldots, v_\ell$ the collection of values advertised by honest parties at the beginning of the phase such that i is missing support on at most λ indices in total. We pair up iterations $r-1$ and r for each even iteration r of the phase. Denote by $I_r \subseteq [\ell]$ the set of indices $j \in [\ell]$ for which i (i) does not yet have a certificate for v_j at the beginning of iteration $r-1$ and (ii) i gains enough new support to cut the missing elements for a certificate for v_j in half during iterations $r-1$ and r.

We consider the r-th pair *good,* if (i) $\sum_{j \in I_r} n_j \geq n/10$ or (ii) i completes a certificate that it did not hold before; otherwise, it is *bad.* Observe that since

for each v_j, the number of times the missing support can be cut in half until i obtains a certificate for v_j is $\lceil \log(t+1) \rceil = O(\log n)$, and $\ell \leq 2n$, within $O(\log n)$ good iteration pairs, i obtains two certificates.

Now consider a bad iteration pair. Note that in the second iteration of the pair, i will call $\mathcal{F}_{\mathsf{pull}}$ in Step 5 with input A'_i which contains all the elements i had obtained by the beginning of iteration $r-1$. Moreover, by assumption there are at least $n/5$ honest parties that advertise two values since there beginning of the phase. So long as i has not yet obtained two certificates, each of these parties advertises at least one value for which i does not hold a certificate.

We iterate over these parties in the order in which the execution determines their responses to i in the call to $\mathcal{F}_{\mathsf{pull}}$ to which they input the certificate C_b, $b \in \{0,1\}$, which i does not yet hold. If the iteration pair is bad, more than $n/5 - n/10 = n/10$ of these responses satisfy that for at least half of the indices on which C_b supports the value it certifies, but A'_i does not, i has not pulled an element supporting the value in the previously evaluated responses or iteration $r-1$: otherwise, we had that $\sum_{j \in I_r} n_j \geq n/10$. Note that this holds independently of which parties the adversary corrupts, since $\mathcal{F}_{\mathsf{pull}}$ does not leak whether i pulls a new element supporting the value before the call to $\mathcal{F}_{\mathsf{pull}}$ completes.

We claim that during a bad iteration pair, due to the above process i obtains at least $n/10$ times with a probability independently bounded from below by $\lambda/(n\log(n))$ one of the initially at most λ elements it is missing to complete the certificates $v_1, ..., v_\ell$. To see this, note that for each $\gamma \in [c]$,

1. in the relevant call to $\mathcal{F}_{\mathsf{pull}}$, for any party i' advertising a certificate containing *some* index on which A'_i does not support the same value, $R_i[\gamma][i']$ contains such an index with probability at least $\lambda/(n\log(n))$,
2. if for at least half of such indices, i did not yet obtain an element, the probability for a new one to come earlier in $R_i[\gamma][i']$ than any already obtained one is at least $1/2$,
3. we ensured that the equality test succeeds with probability at least $1 - 1/n$, and
4. events (i)–(iii) are independent and in conjunction imply that i pulls one of the λ missing elements with probability at least $\lambda/(2n\log(n)) - 1/n$.

Since this happens independently for each γ and $\lambda \gg \log(n)$, we can infer a lower bound of $\lambda/(n\log(n))$ on the probability to pull support for the respective value on some new index.

To complete the proof, we now follow the above process for each iteration pair, determining whether it is good or bad and keeping tally of the number of relevant elements obtained during bad iterations. By the above discussion, in expectation each bad iteration contributes at least $n/10 \cdot \lambda/(n\log(n)) = \lambda/(10\log n)$ such elements. As the probability lower bound of $\lambda/(n\log(n))$ holds independently for each considered event, by Theorem 5(ii) with $\delta = 1/2$ within $20\log n$ bad iteration pairs, i learns λ such elements w.h.p. However, this entails that i obtains certificates for all values $v_1, ..., v_\ell$.

On the other hand, recall that there can be only $O(\log n)$ good iteration pairs until i obtains two certificates. As each iteration pair is either good or bad, we conclude that i obtains two certificates within $O(\log n)$ iterations in total.

Lemma 25. *An honest party sends $O(n^2(\ell + \kappa))$ bits in a single invocation of* Pull*.*

Proof. In an invocation of Pull, an honest party sends a message to every other party containing two public keys. In addition, they send to each party at most one request message and at most one response message. A request message contains a constant number of arrays of $\lceil \frac{\lambda}{\log n} \rceil$ pairs of indices (of length $\log n$) and information for equality testing (of length $\log \ell$). Each response message contains a constant number of elements. Finally, observe that

$$O\left(n^2\left(\ell + \kappa + \frac{\lambda(\log(n) + \log(\ell))}{\log(n)}\right)\right) = O(n^2(\ell + \kappa)),$$

since $\lambda \leq \kappa$ by model assumption and either $\ell \in n^{O(1)}$ or $\lambda \log(\ell) \leq n \log(\ell) \in O(\ell)$.

Lemma 26. *An iteration of* DistributeCertificate *where $\mathcal{F}_{\mathsf{pull}}$ is implemented by* Pull *takes $O(1)$ communication rounds.*

Proof. An iteration of DistributeCertificate invokes $\mathcal{F}_{\mathsf{pull}}$ a constant number of times. Each invocation of Pull requires 3 communication rounds, one for sending each set of ReportKey, Request, and Response messages.

Wrapping Up

Proof (Proof of Theorem 3). To see property (i), observe that honest parties executing $\mathcal{F}_{\mathsf{pull}}$ never create new signatures, which extends to Distribute Certificate. To see property (ii), observe that by Lemma 26, each iteration takes $O(1)$ communication rounds. Hence, we can apply Corollary 7 to see that certificates become widely known, and Lemma 23 or Lemma 24 depending on which case of Lemma 22 applies. The communication complexity bound readily follows from Lemma 25 and the round complexity bound.

We now complete the missing proof of Lemma 1.

Proof (of Lemma 1). We run Distribute Certificate $\lceil \lambda/n \rceil$ times concurrently and independently (all with the given inputs), where we replace the parameter λ in the theorem by $\Theta(n)$. Parallel invocation does not invalidate the protocol's security properties for the following reason. An execution of DistributeCertificate is fully governed by this initial state, the actions of the adversary, and the randomness DistributeCertificate uses (via $\mathcal{F}_{\mathsf{pull}}$). The latter is independent between different executions of DistributeCertificate, implying an information-theoretic impossibility for the parallel invocations to reveal any information about each other beyond their initial state. Since Theorem 3 requires no assumptions on the

initial states of the parties being secret, we may w.l.o.g. assume that the adversary has full knowledge of them. Accordingly, we can apply Theorem 3 to each of the parallel runs, resulting in independent probability of failure of $2^{-\Omega(n)}$ for each of them.

Note that the approach preserves the running time and property (i) (as it holds deterministically), and increases the bit complexity by a factor of $\lceil \lambda/n \rceil$, resulting in $O(n\lambda(\ell+\kappa)\log^2(n)))$ bits sent in total. We decree that parties output up to two of the certificates they obtained, breaking ties arbitrarily. Observe that this ensures that property (ii) is violated if and only if it is violated in all parallel calls to DistributeCertificate, which happens with probability $2^{-\Theta(n\lceil\lambda/n\rceil)} = 2^{-\Omega(\lambda)}$, as required.

E Missing Proofs of Section 4

In this appendix we prove the correctness and analyze the communication of our BinaryConsensus, GradedConsensus, CollectiveBC and Parallel BC protocols.

E.1 Binary Consensus

Proof of Lemma 2. We prove the statement by induction on $|K|$, where the base case of $|K| = 1$ is trivial. Hence, assume in the following that $|K| = k$ and the statement holds for any K with $|K| < k$.

First, we prove that if there is $b \in \{0,1\}$ so that all honest parties P_i satisfy $v_i = b$ at the beginning of a loop iteration, then no honest party modifies v_i. If the precondition of this claim holds, by the validity of GradedConsensus all honest parties P_i output $(b, 2)$ in Step 1. Thus, regardless of the output of the first recursive call to BinaryConsensus, no honest party changes v_i from b. Note that the recursive call completes, since we enforce fixed running times from all our protocols and subroutines. Validity immediately follows from this claim.

Concerning agreement, observe that either K_1 or K_2 must have a minority of corrupted parties. Let $j \in \{1,2\}$ be such that K_j has a minority of corrupted parties and consider the j-th loop iteration. If some honest party outputs $(b, 2)$ for some $b \in \{0,1\}$ from the invocation of GradedConsensus in Step 1, by the graded agreement property of GradedConsensus, all honest parties in K_j input b to the invocation of BinaryConsensus in Step 2. By the induction hypothesis, the recursive call satisfies validity. Hence, all honest parties in K_j output b from BinaryConsensus, and all honest parties $P_i \in K$ end Step 3 with $v_i = b$. On the other hand, in the case that no honest party outputs grade 2 from the first invocation of GradedConsensus, all honest parties set v_i to the same value $b \in \{0,1\}$ in Step 3, by agreement of the recursively invoked BinaryConsensus instance of Step 2. Either way, applying the claim once more, it follows that all honest parties output the same value.

Proof of Lemma 3. The communication complexity of BinaryConsensus is given by the following recurrence, where k is the number of parties in an instance and $GC(k)$ is the communication complexity of GradedConsensus for k parties.

$$C(k) = \left\{ \begin{array}{ll} O(\kappa), & \text{if } k \leq c \\ O(GC(k)) + C(\lceil k/2 \rceil) + C(\lfloor k/2 \rfloor) + O(k^2), & \text{otherwise} \end{array} \right\}$$

We show the bound for n and λ being powers of 2; this is sufficient to show the asymptotic bounds for the general case.

From Lemma 5, we have that $GC(k) = O(\kappa k^2 + DC)$, where DC is the communication complexity of DistributeCertificate. We separate the calculation into two cases.

Case 1: $\lambda \geq cn$ for a sufficiently small constant $c > 0$. In this case, we use the $O(n\lambda(\ell + \kappa)\log^2(n))$ upper bound on the communication complexity of DistributeCertificate from Lemma 1, where $\ell = 1$ for binary consensus. We get:

$$\begin{aligned} C(n) &\leq n \cdot O(\kappa) + \sum_{i=0}^{\log(n)} 2^i \cdot O(\kappa(2^{-i}n)^2 + \lambda\kappa\log^2(n) \cdot 2^{-i}n) \\ &= O(n\kappa) \cdot \sum_{i=0}^{\log(n)} 2^{-i}n + \lambda\log^2(n) \\ &= O(n\kappa(n + \lambda\log^3(n))) \\ &= O(n\lambda\kappa\log^3(n)), \end{aligned}$$

where the last step uses that $n = O(\lambda)$.

Case 2: $\lambda < cn$. In this case, we use the $O(n^2(\ell+\kappa)\log^2(n)/\log(n/\lambda))$ upper bound (with $\ell = 1$) on the communication complexity of DistributeCertificate from Theorem 3 until the number of parties in the recursive call is small enough for the first case to apply, i.e., the size of the sets in a recursive call becomes λ. This yields

$$\begin{aligned} C(n) &= O\left(\frac{n}{\lambda} \cdot \lambda^2\kappa\log^3(n) + \sum_{i=0}^{\log(n/\lambda)-1} 2^i \cdot \left(\kappa(2^{-i}n)^2 + (2^{-i}n)^2\kappa \cdot \frac{\log^2(n/2^i)}{\log(n/(2^i\lambda))}\right)\right) \\ &= O\left(n\lambda\kappa\log^3(n) + n^2\kappa\log^2(n) \sum_{i=0}^{\log(n/\lambda)-1} \frac{1}{2^i\log(n/(2^i\lambda))}\right) \\ &= O\left(n\lambda\kappa\log^3(n) + n^2\kappa\log^2(n) \sum_{i=0}^{\log(n/\lambda)-1} \left(\frac{1}{\sqrt{2}}\right)^i \cdot \frac{1}{(\sqrt{2})^i(\log(n/\lambda) - i)}\right) \\ &= O\left(n\lambda\kappa\log^3(n) + \frac{n^2\kappa\log^2(n)}{\log(n/\lambda)}\right), \end{aligned}$$

where the last step uses that $(\sqrt{2})^i(k-i)$ for $i \in \{0, ..., k-i\}$ is, for k being at least a sufficiently large constant, uniformly bounded from below by k.

E.2 Graded Consensus

Proof of Lemma 4. We first prove that the protocol satisfies graded validity. If all of the honest parties have the same input x, and no certificate of size $t+1$ can be created for a different value, all honest parties receive a certificate of size $t+1$ for x and set $v = x$ and $g = 2$. A certificate of length $t+1$ for a value other than x never exists over the course of the protocol, and thus is never learned by any honest party. As a result, all honest parties output $(x, 2)$.

Next, we prove that the protocol satisfies graded agreement. Assume that some honest party p outputs $(v_p, 2)$. Party p must have adopted v with grade 2 prior to any invocations of DistributeCertificate (this is the only time a party sets $g = 2$). At the same time, p must have added the certificate for v_p to its set $\mathcal{C}$. By the properties of DistributeCertificate, all honest parties learn of the certificate for v_p after the first invocation of DistributeCertificate. Since p didn't change $v = \perp$, or $g = 1$ after either of the two invocations of DistributeCertificate, by the properties of DistributeCertificate, no honest party must have known of a certificate of size $t + 1$ for a value $v' \neq v$ prior to either invocation of DistributeCertificate. Then it must be the case that all honest parties set $v = v_p$ and $g = 1$ after the first invocation of DistributeCertificate (if they didn't already have $v = v_p$). As a result, prior to the second invocation of DistributeCertificate, all honest parties have $v = v_p$ and $g \geq 1$. If any honest party learns of a certificate for a value other than v_p through the second invocation of DistributeCertificate, they don't change their value v, and only parties with $g = 2$ change their grade to 1. As a result, all honest parties output (v, g) s.t. $v = v_p$ and $g \in \{1, 2\}$.

Proof of Lemma 5. The lemma follows from the fact that the protocol consists of a step in which each party multicasts their input v of length κ along with a signature on v followed by two invocations of DistributeCertificate.

E.3 Collective Broadcast

The correctness of our CollectiveBC algorithm, Lemma 6 follows directly from the next two lemmas.

Lemma 27. *The* CollectiveBC *protocol in Figure 8 satisfies validity.*

Proof. Recall that the validity property only imposes a constraint if S has a majority of honest parties, fewer than half of the set of participants K are corrupt, and all honest parties in S have the same input m. If this is the case, then every party $P_i \in K$ receives a codeword symbol cw_i and corresponding witness wit_i along with a single accumulation value acc^* on the encoding of m from a majority of the parties in S, such that Verify(ak, acc^*, cw_i, wit_i)=1, and echoes it. In particular, all honest parties in K input the same accumulator acc^* to GradedConsensus. By the validity property of GradedConsensus all honest parties output $(acc^*, 2)$. It follows that in Step 3, each honest party meets the preconditions to call the decode function on its list of $|K|$ verified codewords symbols (with up to t blanks) $cw_1, ..., cw_{|K|}$, yielding $m^* = \mathsf{Decode}(cw_1, \ldots, cw_{|K|})$.

By the assumption of idealized collision-freeness, no honest party uses invalid symbols, so the call succeeds and outputs the same value m^*, which satisfies $m^* = m$. As a result, all honest parties input 1 to BinaryConsensus and output 1 from BinaryConsensus by validity. Observing that each honest party uses input 1 to the call to binary consensus, which by its validity property ensures output 1, analogous arguments show that each honest party reconstructs and outputs m in the final two steps.

Lemma 28. *The* CollectiveBC *protocol in Figure 8 satisfies agreement and liveness.*

Proof. Recall that these properties need to hold as long as fewer than half of the participating parties K are corrupted, regardless of how many parties in the sending set S are corrupted. Observe that regardless of what messages are sent by the parties in S, all honest parties in K input to GradedConsensus. By the liveness property of GradedConsensus, all honest parties output a pair (v, g). Subsequently, all honest parties input to BinaryConsensus. By the liveness and agreement properties of BinaryConsensus, all honest parties output the same value from BinaryConsensus. If this value is 0, they all return 0^n, i.e., agreement and liveness of the overall protocol hold. If this value was 1, then by the validity property of BinaryConsensus, some honest party must have used input 1. For this to happen, this party must have had output grade 2 from GradedConsensus. By graded agreement, all honest parties must then have output the same accumulator acc^* from graded consensus. Moreover, the party using input 1 for BinaryConsensus must also have a list of $|K|$ codeword symbols $cw_1, \ldots, cw_{|K|}$ (such that at most t are blank) and for the non-blank symbol cw_j have corresponding witnesses wit_j such that $\mathsf{Verify}(ak, acc^*, cw_j, wit_j)$=1. By the idealized collision-freeness of the accumulator, this entails that it can decode $m^* = \mathsf{Decode}(cw_1, \ldots, cw_{|K|})$. This party is then able to (re-)encode m^* as $cw_1, \ldots, cw_{|K|} \leftarrow \mathsf{Encode}\ (m^*)$ to obtain $|K|$ symbols as well as create their witnesses as $wit_j \leftarrow \mathsf{CreateWit}(ak, acc^*, cw_j)$. Accordingly, it sends each party P_j their corresponding witness and codeword, enabling every party to obtain $t + 1$ valid codewords and witnesses, decode m^* as stated, and subsequently return m^*. By the idealized collision freeness of the accumulator, all honest parties return the same m^*, proving agreement and liveness also in this case.

We can now complete the analysis of the communication complexity of CollectiveBC, stated in Lemma 7.

Proof (Proof of Lemma 7). Recall that by our choice od code parameters, each codeword symbol is of size $O(\ell/|K| + \log(n))$, witnesses are of size $O(\kappa \log(n))$, and the accumulator is of size $O(\kappa)$. Each party sends to each other party $O(1)$ symbols, witnesses, and accumulators, for a total of $O(|K|\ell + |K|^2 \kappa \log(n))$ bits.

In addition, there is one call each to GradedConsensus and BinaryConsensus. By Lemma 5 and Lemma 3, the claim follows.

E.4 Parallel Broadcast

Proof of Lemma 8. First we show that the protocol satisfies agreement and validity. It is clear that prior to the invocation of the recursive parallel broadcast protocol, the set V assembled by each honest party contains the input of every other honest party. We use a proof by induction to prove that the recursive protocol satisfies validity and agreement. We prove the statement that if all honest parties in K start RecursiveParallelBC (Figure 10) with an array V satisfying validity, and K has minority corruption, then all honest parties in K output a list V^* satisfying agreement and validity under the induction hypothesis that the protocol satisfies validity and agreement for $\lceil |K|/2 \rceil$ and $\lfloor |K|/2 \rfloor$ parties under minority corruption.

It is clear that the statement holds for the base case when there is a single party in K. We assume that this holds for balanced subsets K_0 and K_1 of K and prove that it holds for K. Note that since K has minority corruption, it must be the case that K_0 or K_1 has minority corruption. Let $j \in \{0, 1\}$ be such that K_j has minority corruption. Then by the assumption, it must be the case that the list V' returned for all honest parties in K_j by the recursive call satisfies validity and agreement. This list is then given by all honest parties in K_j as input to CollectiveBC. By the validity of CollectiveBC, the list V_j returned by all honest parties in K must be V'. Next, consider the parallel invocation of the set K_{1-j}. Here, regardless of what is output from the recursive call of RecursiveParallelBC for K_{1-j}, the agreement property of CollectiveBC ensures that all honest parties in K output the same list V_{1-j}. As a result, each honest node holds exactly the same V_j and V_{1-j}, and after the lists are merged, all honest parties return the same merged-list V^*, guaranteeing agreement.

Since the adversary cannot forge signatures for honest parties, for every index corresponding to an honest party P_j, the list V_{j-1} obtained by every honest party contains either p_j's actual message or is lacking a valid signature. As a result, for each honest party j, $V^*[j]$ is set to P_j's message, proving validity.

The protocol satisfies validity and agreement by the fact that the network (and therefore the set K at the highest level of the recursion tree) satisfies minority corruption.

Finally, we show that the protocol satisfies liveness. First, observe that the CollectiveBC protocol terminates after a finite (polynomial) number of synchronous rounds. There must be a path from the leaf nodes to the root of the recursion tree such that the parties participating in each node on the path have minority corruption. Liveness therefore follows from the liveness of CollectiveBC and the fact that each recursive step terminates after a polynomial number of synchronous rounds.

Proof of Lemma 9. We show the claim for n being a power of 2; the asymptotic cost in the general case is identical. The initial multicast of the inputs incurs a cost of $n^2(\ell + \kappa)$. The communication complexity of recursive parallel broadcast is given by the following recurrence, where k is the number of parties in an instance, $RB(k)$ is the communication complexity of RecursiveParallelBC for k

parties, and $CB(k)$ is the communication complexity of $\mathsf{CollectiveBC}$ for k parties with an input of size $O(n(\ell+\kappa))$, i.e., a vector of n signed messages:

$$\begin{aligned} &RB(k) = 2RB(k/2) + O(CB(k)) \\ &= 2RB(k/2) + O\left(k(n(\ell+\kappa) + \lambda\kappa\log^3(n)) + k^2\kappa \cdot \frac{\log^2(n)}{\max\{\log(k/\lambda), 1\}}\right); \end{aligned}$$

here, the second step applies Lemma 7.

We claim that for $j \in \mathbb{N}$,

$$RB(2^j) = O\left(j2^j(n(\ell+\kappa) + \lambda\kappa\log^3(n)) + \sum_{i=1}^{j} \frac{2^{2i}\kappa\log^2(n)}{\max\{\log(2^i/\lambda), 1\}}\right).$$

We show this by induction on j, where the base case of $j = 0$ is trivial, since $RB(2^0) = RB(1) = 0$. As for the step from $j-1$ to j, the induction hypothesis and the above formula yields that

$$\begin{aligned} &RB(2^j) \\ &= 2RB(2^{j-1}) + O\left(2^j(n(\ell+\kappa) + \lambda\kappa\log^3(n)) + 2^{2j}\kappa \cdot \frac{\log^2(n)}{\max\{\log(2^j/\lambda), 1\}}\right) \\ &= O\left(j2^j(n(\ell+\kappa) + \lambda\kappa\log^3(n)) + \sum_{i=1}^{j} \frac{2^{2i}\kappa\log^2(n)}{\max\{\log(2^i/\lambda), 1\}}\right), \end{aligned}$$

completing the induction. Observe that

$$\frac{2^{2(i+1)}}{\max\{\log(2^{i+1}/\lambda), 1\}} = \frac{4 \cdot 2^{2i}}{\max\{\log(2^i/\lambda) + 1, 1\}} \geq 2 \cdot \frac{2^{2i}}{\max\{\log(2^i/\lambda), 1\}},$$

so the sum is bounded by the product of its final term with a geometric series and we can simplify further, yielding that

$$RB(2^j) = O\left(j2^j(n(\ell+\kappa) + \lambda\kappa\log^3(n)) + \frac{2^{2j}\kappa\log^2(n)}{\max\{\log(2^j/\lambda), 1\}}\right).$$

Plugging in $j = \log n$, the claim of the lemma follows. □